Highlights

Latent Graph Deliberation Networks for Language Model Collaboration

- We reframe language model collaboration as latent graph deliberation, moving beyond one-pass dense communication.
- LM-LGD organizes shared-backbone LM nodes into a reciprocal graph with multi-round hidden-state message passing.
- Receiver-conditioned trust aggregation replaces indiscriminate message summation and curbs hidden-state distribution shift.
- Delayed sparse reporting keeps deliberation dense but commits only selected states to a core decision node.
- LM-LGD improves over LMNet on multiple reasoning/coding benchmarks and on supervision-efficient adaptation across different LM backbones.

Latent Graph Deliberation Networks for Language Model Collaboration

Abstract

Large language models are powerful standalone predictors, yet many hard reasoning and collaboration tasks benefit from organizing computation beyond a single model invocation. Language Model Networks (LMNet) instantiate this direction by treating pretrained language models as reusable nodes that communicate through *dense hidden states* rather than discrete text, enabling end-to-end differentiable collaboration. However, existing LMNet-style graphs remain largely one-pass: messages flow through a fixed feed-forward topology, incoming messages are aggregated by simple summation, and the final prediction is produced without reciprocal deliberation. Once dense communication is available, the central design question becomes how such communication should be *structured*. We propose LM-LGD, an LM-based Latent Graph Deliberation Network, which formulates language model collaboration as multi-round latent deliberation over a Reciprocal LM Graph. In LM-LGD, shared-backbone assistant nodes exchange full-sequence hidden-state messages for several rounds; each receiver computes trust weights for incoming messages and integrates the aggregated message through a gated, normalized latent-state update. After deliberation, a router performs *delayed sparse reporting* by selecting the top-k assistant states, fusing their projected reports, and passing the result to a core decision node. This design mitigates the indiscriminate message accumulation and hidden-state distribution shift that arise from unnormalized summation, while allowing selected deliberated states to inform the final prediction. Across two backbones and multiple benchmarks, LM-LGD improves over LMNet on various LM reasoning tasks under a comparable data and parameter budget, with higher compute (e.g., +4.7 on MATH and +4.3 on GSM8K with Qwen2.5-0.5B), and it yields consistent gains in supervision-efficient adaptation on MMLU, GSM8K, and E2E. These results show latent graph deliberation as a principled alternative to one-pass dense communication: by replacing indiscriminate aggregation with receiver-conditioned trust and delayed sparse reporting, LM-LGD organizes collaboration that is both computationally stable and empirically superior across diverse reasoning tasks.

Keywords: Language model networks, Latent reasoning, Multi-agent collaboration, Hidden-state communication, Graph deliberation, Trust aggregation, Sparse routing

1. Introduction

Large language models (LLMs) have achieved strong performance across language understanding, generation, and reasoning. Yet a single forward pass through one model is often insufficient for tasks that demand specialized knowledge, multi-step reasoning, or sustained computation. A growing body of work therefore improves LLMs not by enlarging a single model, but by *organizing computation around* models: structuring how information is produced, exchanged, and refined before an answer is committed [1, 2, 3].

These efforts span several, largely complementary, directions. Multi-step reasoning and test-time scaling allocate extra computation at inference, through chain-of-thought prompting [1], self-consistency sampling [2], tree- and graph-structured search [4, 5], verifier-guided selection [6], and iterative self-refinement [7]. Text-based multi-agent and workflow systems assign roles to multiple LLMs and support role-based interaction, tool use, or coordinated workflows through natural-language protocols [3, 8, 9], and recent work optimizes the agent topology itself [10]. Latent-reasoning methods instead move intermediate computation into continuous space, replacing explicit token rationales with hidden-state trajectories [11, 12]. These directions indicate that the capability of an LLM-based system depends not only on the parameters of a single model, but also on *how computation around the model is organized*.

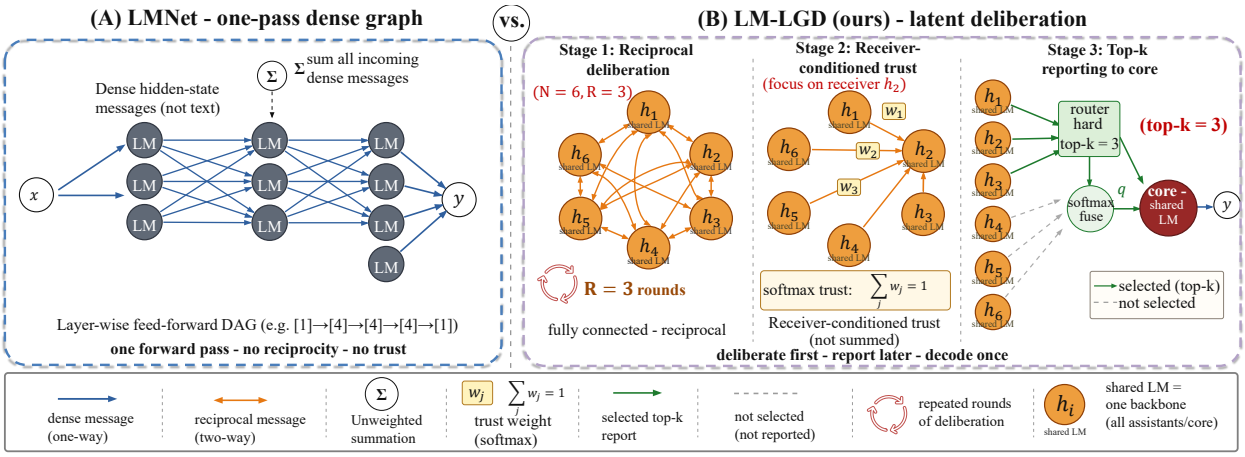


Figure 1: **From one-pass dense routing to latent deliberation.** *Left:* LMNet organizes language-model nodes as a one-pass, layer-wise feed-forward graph; each node *sums* its incoming dense hidden-state messages and runs the shared backbone once, with no reciprocity and no trust allocation. *Right:* LM-LGD turns the same dense channel into a fully connected Reciprocal LM Graph that deliberates for several rounds; each receiver assigns trust weights to incoming messages and updates its latent state through a gate. After deliberation, a router selects a sparse top-k set of deliberated states for a core decision node. LMNet enables differentiable dense LM communication, whereas LM-LGD structures that communication as multi-round deliberation.

Motivated by the need to organize computation beyond a single model, Wu et al. [13] proposed Language Model Networks (LMNet), an LM networking framework that treats each pretrained LM as an atomic unit of computation and optimization in a graph. In LMNet, the collective behavior of LM nodes is shaped by topology, communication, and end-to-end task supervision. LMNet replaces discrete natural-language messages with *dense hidden-state messages*. By removing the embedding and de-embedding steps between intermediate nodes, LMNet lets language-model nodes exchange continuous hidden states directly and learn the communication modules end-to-end from task supervision. This formulation demonstrates that language-model nodes can communicate through machine-native dense vectors in a fully differentiable manner.

Dense communication, however, does not by itself imply effective collaboration. The original LMNet adopts a layer-wise, fully connected *feed-forward* graph: information flows strictly from one layer to the next, and each node feeds the *unnormalized sum* of all incoming dense messages into a shared backbone. Three limitations follow. First, the computation is essentially *one-pass*: a node never revisits or revises its message in light of what its peers conclude, so the graph cannot express the back-and-forth correction that hard tasks often require. Second, *summation treats every sender as equally trustworthy*; there is no input-dependent allocation of importance among incoming messages, and the summed signal can grow in scale with the number of predecessors. Third, and more subtly, because all nodes *share one pretrained backbone*, that backbone is forced to process hidden states whose scale and direction may drift far from the distribution it was pretrained on, an aggregation-induced *hidden-state distribution shift* that is strongest precisely at the multi-input intermediate and output nodes. Thus, LMNet establishes dense communication as a medium, while the organization of that communication remains weakly specified.

Thus, the limitations above motivate us to the design objective addressed in this work:

> *What mechanism enables dense shared–backbone LM nodes to deliberate in latent space before producing a final prediction?*

We believe that simply adding more LMNet layers or more dense edges would retain the one-pass character of the computation and can further compound the distribution shift described above. So, our design preserves the differentiable advantage of dense hidden-state communication while enabling *reciprocal*, *trust-aware*, and *multi-round* collaboration, and keeps the shared backbone close to its native input distribution.

Specifically, we propose an LM-based Latent Graph Deliberation Network (LM-LGD) framework (see

Figure 1, right). LM-LGD builds a Reciprocal LM Graph in which N assistant LM nodes share a single stripped backbone but are differentiated by directed edge modules, zero-initialized role embeddings, and divergent communication histories. Given an input, the assistants defer reporting and first deliberate for R rounds of full-neighbor message passing. In each round, every receiver collects dense messages that preserve the full token sequence from the other assistants and computes *receiver-conditioned trust aggregation* weights conditioned on the receiver's own state, the message content, the input context, and the sender/receiver identities. The receiver then aggregates messages with a softmax over senders, so the weights sum to one rather than accumulating without bound, and integrates the result through a gated, layer-normalized residual update before passing it through the shared backbone.

After deliberation, LM-LGD performs *delayed sparse reporting*. A router assigns a relevance score to each deliberated assistant, a hard top-k operator selects the most useful states, and their reports are fused by a softmax over the selected set and combined with a core node through a second gated residual. The core node then produces the final logits. This design keeps communication *dense and full-neighbor during deliberation* so information is thoroughly exchanged, while using *sparse reporting at the moment of decision* so the core node receives only selected, high-value information. This reporting step uses delayed hard selection; differentiable relaxations of top-k routing are left for future work.

Our contributions are summarized as follows.

- We identify an organization problem in dense language-model graphs: differentiable hidden-state communication specifies how nodes exchange messages, but not how they revise those messages, allocate trust, or decide what should reach the final predictor. Motivated by this observation, we formulate *latent graph deliberation* as an organizing principle for language model collaboration.
- We propose LM-LGD, a Reciprocal LM Graph framework in which shared-backbone assistant nodes exchange full-sequence hidden states and revise one another through multi-round latent message passing before a core node commits to the final prediction.
- We design receiver-conditioned trust aggregation and delayed sparse reporting to make dense communication both trust-aware and decision-efficient: receiver-conditioned trust aggregation replaces unnormalized summation with receiver-conditioned normalized aggregation, while delayed sparse reporting keeps deliberation dense and makes only the final reporting step sparse through a hard top-k selection, reducing hidden-state distribution shift.
- We demonstrate the effectiveness of trust-guided latent deliberation across general-capability and supervision-efficient adaptation benchmarks, and use mechanistic analyses and ablations to show that the gains come from better communication organization rather than added graph capacity alone. Code is available in `https://anonymous.4open.science/r/LM-LGD-C11D/`.

The remainder of this paper is organized as follows. Section 2 reviews related work; Section 3 formalizes language-model nodes, dense messages, and the deliberation setting; Section 4 presents LM-LGD; Sections 5 and 6 report main results and ablations; Section 7 discusses scope and implications; and Section 8 concludes.

2. Related Work

2.1. Networking Pretrained Language Models

Recent work has increasingly treated pretrained language models as components in larger computational systems. These methods can be categorized by the communication mechanism used among the component models. In text-based systems, multiple language models communicate through natural-language messages. Chain-of-thought prompting elicits intermediate reasoning steps within a single model [1], while self-consistency and best-of-N sampling draw multiple candidate traces and aggregate their answers [2]. Multi-agent debate uses interactions in which models critique and revise one another's outputs [3], ReAct interleaves reasoning with tool actions [8], and MetaGPT assigns human-designed roles to coordinate a workflow [9]. A related set of methods optimizes the system structure directly: tree- and graph-structured search

explores partial solutions [4, 5], and GPTSwarm represents an agent system as an optimizable graph and searches over its topology [10]. These approaches indicate that topology and communication protocol affect the capability of multi-model language systems. Their communication, however, remains discrete natural language, which introduces long intermediate traces and makes the protocol difficult to train directly from final-task supervision.

Another line of work explores communication in continuous latent space. Latent-reasoning methods represent intermediate computation with hidden states rather than token-level rationales: Coconut reuses the final hidden state of one reasoning step as the embedding input to the next step [11], and CODI distills an explicit chain of thought into a continuous trajectory through self-distillation [12]. Most directly, Language Model Networks (LMNet) connect pretrained models as reusable nodes that communicate through dense hidden states [13]. In LMNet, stripped transformers serve as vertex modules, and trainable sequence-to-sequence modules serve as communication edges. Intermediate nodes exchange dense vectors, and the network is trained end-to-end while retaining a natural-language interface at the boundary. LMNet is the closest prior work to ours, and we adopt its dense communication medium. We depart from LMNet in the structure imposed on this communication. LMNet arranges nodes as a one-pass, layer-wise feed-forward graph and sums the messages entering each node, so a node never revises its message using feedback from its peers and every sender receives the same treatment. We study latent deliberation with trust allocation and selective commitment before the final answer.

2.2. Deliberation and Message Passing over Time

Our method is related to two established mechanisms for refining information across computation steps. The first is deliberation in sequence generation. Deliberation networks address the limits of one-pass decoding by using a first-pass decoder to produce a draft and a second-pass decoder to revise it with the full draft as context [14]. The connection to our work is the use of additional computation to revise an intermediate representation. The setting is different: prior deliberation revises generated sequences after decoding, whereas LM-LGD deliberates over hidden-state messages before the final decoding step, and its deliberating units are connected language-model nodes rather than decoder passes over a sentence.

The second mechanism is iterative message passing over graphs. Graph neural networks repeatedly update each node from the messages of its neighbors, and graph attention learns how much weight to assign to each neighbor rather than averaging them uniformly [15, 16]. Mixture-of-experts models provide a related form of sparse routing by sending each input to a small number of experts for efficiency and specialization [17, 18]. LM-LGD adapts these mechanisms to a network of language models rather than to ordinary feature vectors. Its nodes are stripped transformers [19], its messages are full-sequence hidden states of shape $[B, T, d]$ rather than low-dimensional node features, and its routing is a graph-level selection applied after several rounds of deliberation rather than a per-token expert choice. The resulting architecture combines trust-weighted multi-round message passing with delayed sparse reporting, distinguishing it from one-pass dense communication and from direct applications of graph attention or mixture-of-experts routing.

3. Preliminaries and Task Definition

We now formalize language-model nodes, dense hidden-state messages, the one-pass LMNet computation, and the latent graph deliberation setting that LM-LGD targets.

3.1. Language Model as a Reusable Node

Given a tokenized text $x = (x_1, \ldots, x_T)$, a standard causal language model first maps tokens to continuous representations through an embedding layer,

$$\mathbf{X} = \mathrm{Emb}(x), \qquad \mathbf{X} \in \mathbb{R}^{B \times T \times d}, \tag{1}$$

where B is the batch size, T the sequence length, and d the hidden size. A transformer backbone F_θ then maps the input hidden sequence to an output of the same shape, $\mathbf{H} = F_\theta(\mathbf{X})$, and a language-model

head projects hidden states to vocabulary logits. With $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{|V|\times d}$, we write $\mathbf{Z} = \mathbf{H}\mathbf{W}_{\text{lm}}^{\top} \in \mathbb{R}^{B\times T\times |V|}$. In a language model network, we view the transformer backbone, stripped of its external embedding/de-embedding boundary, as a *reusable LM node* $F_\theta : \mathbb{R}^{B\times T\times d} \to \mathbb{R}^{B\times T\times d}$. Such a node consumes a dense hidden-state sequence rather than a natural-language string, so intermediate communication between nodes can remain in continuous hidden-state space.

3.2. Dense Hidden-State Messages

Let $\mathbf{h}_i \in \mathbb{R}^{B\times T\times d}$ denote the hidden state of node i. A directed communication edge $E_\phi^{i\to j}$ transforms the sender's hidden state into a receiver-specific dense message,

$$\mathbf{m}_{i\to j} = E_\phi^{i\to j}(\mathbf{h}_i)\,, \qquad \mathbf{m}_{i\to j} \in \mathbb{R}^{B\times T\times d}. \tag{2}$$

The message is neither a pooled vector nor token logits, but a full sequence-level dense tensor that preserves the token-position dimension. Following LMNet [13], this avoids de-embedding hidden states into text and re-embedding them, keeping communication differentiable.

3.3. One-Pass LMNet Communication

The original LMNet can be abstracted as a layer-wise directed acyclic graph. Nodes at layer ℓ send dense messages to nodes at layer $\ell{+}1$. For a receiver j, the incoming messages are aggregated and passed through the shared backbone,

$$\mathbf{c}_j^{\ell+1} = \sum_{i\in\mathcal{N}^-(j)} E_\phi^{i\to j}\big(\mathbf{h}_i^{\ell}\big)\,, \qquad \mathbf{h}_j^{\ell+1} = F_\theta\big(\mathbf{c}_j^{\ell+1}\big)\,, \tag{3}$$

where $\mathcal{N}^-(j)$ is the set of predecessors. The aggregation in Eq. (3) is an *unnormalized sum*. This makes the communication path differentiable, but introduces two issues that motivate our work. First, computation is one-pass and feed-forward: nodes never exchange messages reciprocally or revise over multiple rounds. Second, the sum treats all predecessors as equally important and can grow in scale with $|\mathcal{N}^-(j)|$, shifting the hidden-state distribution seen by the shared backbone.

3.4. Latent Graph Deliberation Setting

Rather than a generic graph network, LM-LGD considers latent graph deliberation over language-model nodes. Given input hidden states $\mathbf{X}$, we instantiate N assistant nodes and one core node. The assistants first perform R rounds of message passing over a *reciprocal* directed graph; in each round, every assistant receives dense messages from the others and updates its own hidden state. The assistants report to the core node after deliberation. Writing the assistant states at round r as $\mathcal{H}^r = \{\mathbf{h}_1^r, \ldots, \mathbf{h}_N^r\}$ with $\mathbf{h}_i^r \in \mathbb{R}^{B\times T\times d}$, LM-LGD realizes a function

$$f_{\text{LM-LGD}}(x) = \text{Core}\Big(\text{Report}\big(\text{Deliberate}(\text{Emb}(x))\big)\Big), \tag{4}$$

whose output remains usable for standard NLP tasks while its internal communication is carried out entirely by a dense hidden-state graph.

3.5. Training Objectives

LM-LGD is trained from final-task supervision only. For causal language modeling, the core node emits logits $\mathbf{Z} \in \mathbb{R}^{B\times T\times |V|}$ and we minimize the standard autoregressive negative log-likelihood,

$$\mathcal{L}_{\text{lm}} = -\sum_t \log p_\Theta(x_t \mid x_{<t})\,. \tag{5}$$

For sequence classification or multiple-choice tasks such as MMLU, we pool the core node's final hidden states, e.g. the last valid token, $\mathbf{u} = \text{pool}(\mathbf{H}_{\text{core}}) \in \mathbb{R}^{B\times d}$, apply a classification head $\mathbf{o} = \mathbf{W}_{\text{cls}}\mathbf{u}$, and minimize cross-entropy $\mathcal{L}_{\text{cls}} = -\log p_\Theta(y \mid x)$. Importantly, the messages, trust weights, router decisions, and intermediate hidden states are never supervised directly: the communication protocol is learned solely from the final objective.

3.6. Problem Statement

We can now state the problem precisely. *The goal is to organize multiple shared-backbone LM nodes so that, while preserving the natural-language input/output interface, they (i) deliberate over multiple reciprocal rounds in hidden-state space, (ii) decide how much to trust, aggregate, and report each node's information, and (iii) learn this entirely from final-task supervision.* This decomposes into three sub-goals: *preserve* dense differentiable communication by keeping messages as $[B, T, d]$ sequences; *organize* communication by replacing unconditional summation with receiver-conditioned trust aggregation and one-pass propagation with multi-round message passing; and *commit selectively* by delaying top-k selection until after deliberation, so exploration stays dense while the final decision stays sparse.

4. LM-based Latent Graph Deliberation Network (LM-LGD)

We build LM-LGD on the dense hidden-state communication introduced by LMNet [13], and reorganize that communication into a two-phase graph computation. Rather than sending messages once through a layer-wise feed-forward topology, we arrange the assistant LM nodes as a Reciprocal LM Graph: assistants exchange directed hidden-state messages for multiple rounds, and only a selected subset reports to the core decision node (Figure 2).

For each input token sequence, we compute shared embeddings to obtain the input hidden states $\mathbf{X} \in \mathbb{R}^{B\times T\times d}$, and initialize N assistant states of the same shape. A forward pass then proceeds in two phases. In the *deliberation phase*, assistants perform R rounds of full-neighbor directed communication. At each round, every receiver assistant collects dense hidden-state messages from the other assistants; we aggregate these messages with receiver-conditioned trust aggregation and update the receiver through a gated latent-state update followed by the shared backbone. In the *reporting phase*, the router scores the deliberated assistants, selects the top-k reports, fuses them, and conditions the core decision node on the fused report before the final logits or task prediction. Algorithm 1 summarizes this two-phase computation. All assistant and core computations reuse a single shared backbone F_θ. The only newly introduced trainable parameters are the communication components: edge modules, trust scorer, router, report projection, gates, and role embeddings.

4.1. Role-Preserving Assistant Initialization

We initialize each assistant from the same input hidden state plus a learnable role embedding,

$$\mathbf{h}_i^0 = \mathbf{X} + \mathbf{p}_i, \qquad \mathbf{p}_i \in \mathbb{R}^d, \tag{6}$$

where $\mathbf{p}_i$ is broadcast across the $[B, T, d]$ hidden states. We store the role embeddings as a parameter matrix of shape $[N, d]$ and *initialize them to zero*. If $\mathbf{p}_i$ were initialized randomly, each assistant would present the shared backbone with inputs displaced from the pretrained embedding distribution from the first training step, which can hinder convergence. Zero initialization guarantees $\mathbf{h}_i^0 = \mathbf{X}$ at the start of training, so every assistant starts from a hidden-state distribution the backbone already handles well. The learned role offsets then allow assistants to specialize gradually under the downstream loss without forcing early deviations from the pretrained hidden-state distribution.

4.2. Reciprocal LM Graph and Directed Edge Messages

During deliberation, we construct a fully connected directed graph over the assistants: for every ordered pair (j, i) with $j \neq i$ there is a directed edge $j \to i$. At round r, sender j sends receiver i the message

$$\mathbf{m}_{j\to i}^r = E_\phi^{j\to i}\big(\mathbf{h}_j^r + \mathbf{s}_j + \mathbf{t}_i\big), \tag{7}$$

where $E_\phi^{j\to i}$ is an LMNet-style sequence-to-sequence edge module and $\mathbf{s}_j, \mathbf{t}_i \in \mathbb{R}^d$ are sender- and receiver-side edge identity embeddings. The message $\mathbf{m}_{j\to i}^r$ remains a sequence-level $[B, T, d]$ tensor. By default, we use *independent directed edges*: each ordered pair has its own edge module, so different communication directions can learn different transformations; a lighter *shared-edge* variant is also supported (Section 6). For parameter efficiency, all assistants and the core node share the single backbone F_θ, so the added cost over LMNet comes from the communication modules, not from replicating the language model.

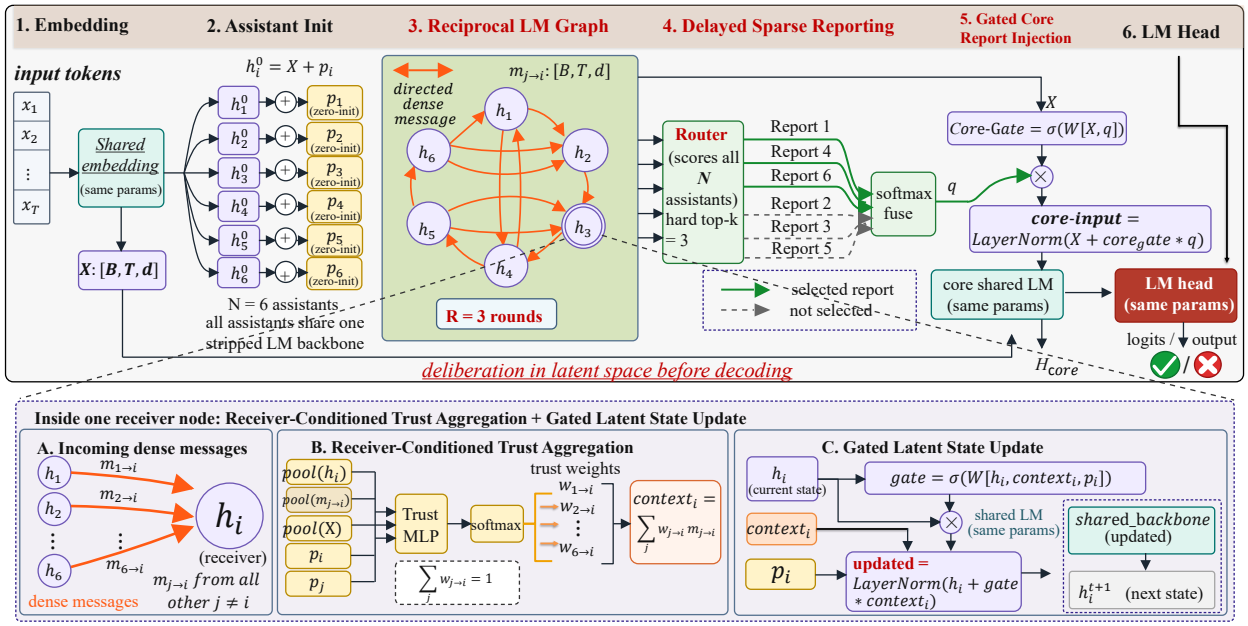


Figure 2: **Overview of LM-LGD.** Input tokens are embedded into $\mathbf{X}$ and used to initialize N assistant nodes, each as $\mathbf{X} + \mathbf{p}_i$ with a zero-initialized role embedding. Assistants exchange full-sequence hidden-state messages over a fully connected Reciprocal LM Graph for R rounds; each receiver computes receiver-conditioned trust aggregation weights over its incoming messages (inset) and integrates them through a gated, layer-normalized residual update before the shared backbone. After deliberation, a router applies delayed sparse reporting: the top-k assistants are selected, their reports are fused by a softmax over the selected set, and the fused report is combined with the core node through a second gated residual. The core node, reusing the same shared backbone, produces the output. A single color denotes reuse of one shared backbone across all nodes.

4.3. Receiver-Conditioned Trust Aggregation

We replace the unnormalized summation in LMNet (Eq. (3)) with *receiver-conditioned trust* aggregation. For receiver i, we assign each incoming message a scalar trust score

$$a^r_{j\to i} = \mathrm{Trust}\Big(\mathrm{pool}(\mathbf{h}^r_i),\ \mathrm{pool}(\mathbf{m}^r_{j\to i}),\ \mathrm{pool}(\mathbf{X}),\ \mathbf{p}_i,\ \mathbf{p}_j\Big), \tag{8}$$

where $\mathrm{pool}(\cdot)$ takes the last valid token's hidden state and Trust is a small multi-layer perceptron. The scores are normalized with a softmax over all senders of i,

$$\alpha^r_{j\to i} = \frac{\exp\big(a^r_{j\to i}\big)}{\sum_{j'\neq i}\exp\big(a^r_{j'\to i}\big)}, \qquad \mathbf{c}^r_i = \sum_{j\neq i} \alpha^r_{j\to i}\,\mathbf{m}^r_{j\to i}. \tag{9}$$

This aggregation differs from summation in two ways. First, the weights sum to one, so the aggregated context is a convex combination of messages and does not inflate in scale with the number of senders. Second, the weights are *input-dependent*: each receiver assigns trust based on its own state, the candidate message, the global input context, and the role identities. We pool hidden states only to *score* messages; the aggregated quantity $\mathbf{c}^r_i$ remains a full sequence-level tensor. Thus, LM-LGD does not reduce each assistant to a compressed node vector before communication or reporting.

4.4. Gated Latent State Update

We do not overwrite the receiver state with $\mathbf{c}_i^r$. Instead, we update each receiver through a gated, normalized residual followed by the shared backbone,

$$\mathbf{g}_i^r = \sigma\big(\mathbf{W}_g\,[\mathbf{h}_i^r;\,\mathbf{c}_i^r;\,\mathbf{p}_i]\big), \tag{10}$$

$$\mathbf{u}_i^r = \mathrm{LayerNorm}\big(\mathbf{h}_i^r + \mathbf{g}_i^r \odot \mathbf{c}_i^r\big), \tag{11}$$

$$\mathbf{h}_i^{r+1} = F_\theta\big(\mathbf{u}_i^r\big). \tag{12}$$

The gate $\mathbf{g}_i^r \in \mathbb{R}^{B\times T\times d}$ controls, per token and per channel, how much external information enters the receiver state; LayerNorm controls hidden-state scale drift; and the shared backbone transforms the integrated latent state. The update therefore includes a backbone computation rather than only a linear interpolation between the previous state and the incoming context.

4.5. Delayed Sparse Reporting

We apply delayed sparse reporting only at the final reporting stage. During deliberation, the assistants communicate without top-k pruning; after deliberation, we introduce sparsity when passing information to the core node. After R rounds, each assistant holds a deliberated state $\mathbf{h}_i^R$, and the router assigns it a score,

$$\rho_i = \mathrm{Router}\Big(\mathrm{pool}(\mathbf{h}_i^R),\,\mathrm{pool}(\mathbf{X}),\,\mathbf{p}_i\Big). \tag{13}$$

The top-k operator selects the highest-scoring assistants, $\mathcal{S} = \mathrm{TopK}(\{\rho_i\}_{i=1}^N, k)$. The selected reports are projected, $\mathbf{q}_i = \mathbf{W}_{\mathrm{rep}}\mathbf{h}_i^R$, weighted by a softmax over the selected set, and fused,

$$\beta_i = \frac{\exp(\rho_i)}{\sum_{i'\in\mathcal{S}}\exp(\rho_{i'})}\ (i\in\mathcal{S}), \qquad \mathbf{q} = \sum_{i\in\mathcal{S}}\beta_i\,\mathbf{q}_i. \tag{14}$$

The top-k operation defines the non-differentiable part of this router, whereas the fusion weights β_i over the selected assistants remain differentiable. The router should therefore be interpreted as delayed hard selection followed by differentiable softmax fusion over the selected reports, rather than as a fully differentiable sparse router.

4.6. Gated Core Report Injection and Output

The core node does not participate in the assistants' rounds; it conditions only on the fused report $\mathbf{q}$ through a gated residual,

$$\mathbf{g}_{\mathrm{core}} = \sigma\big(\mathbf{W}_c\,[\mathbf{X};\,\mathbf{q}]\big), \tag{15}$$

$$\mathbf{X}_{\mathrm{core}} = \mathrm{LayerNorm}\big(\mathbf{X} + \mathbf{g}_{\mathrm{core}} \odot \mathbf{q}\big), \tag{16}$$

$$\mathbf{H}_{\mathrm{core}} = F_\theta\big(\mathbf{X}_{\mathrm{core}}\big). \tag{17}$$

For causal language modeling, with $\mathbf{W}_{\mathrm{lm}} \in \mathbb{R}^{|V|\times d}$, the output logits are $\mathbf{Z} = \mathbf{H}_{\mathrm{core}}\mathbf{W}_{\mathrm{lm}}^\top$; for classification we pool $\mathbf{H}_{\mathrm{core}}$ and apply a task head. We combine the report with the *original* input hidden state rather than replacing it, which keeps the core node anchored to the pretrained distribution while still conditioning it on deliberated information. Algorithm 1 summarizes the full forward pass.

4.7. Training Strategy

We train LM-LGD end-to-end from final-task supervision; the trust weights, router scores, messages, and reports receive no direct labels. To align with LMNet, our main experiments use a two-stage schedule. In the first stage we *freeze* the shared backbone and train only the non-backbone graph/output modules: edge modules, trust scorer, router, report projection, gates, role embeddings, and any untied output or task head. This stage trains the communication components to map dense messages into a hidden space the backbone

Algorithm 1 LM-LGD forward pass

Require: tokens x; assistants N; rounds R; reports k

$\mathbf{X} \leftarrow \mathrm{Emb}(x)$

for $i = 1, \ldots, N$ **do** ▷ role-preserving init; $\mathbf{p}_i$ zero-init

  $\mathbf{h}_i^0 \leftarrow \mathbf{X} + \mathbf{p}_i$

end for

Phase 1: Deliberation *(message passing and state updates)*

for $r = 0, \ldots, R-1$ **do**

  for each receiver i **do**

    compute $\mathbf{m}_{j\to i}^r$ for all $j \neq i$ ▷ Eq. (7)

    $\alpha_{j\to i}^r \leftarrow$ softmax of trust scores ▷ Eq. (8)–(9)

    $\mathbf{c}_i^r \leftarrow \sum_{j\neq i} \alpha_{j\to i}^r \mathbf{m}_{j\to i}^r$

    $\mathbf{h}_i^{r+1} \leftarrow F_\theta\big(\mathrm{LayerNorm}(\mathbf{h}_i^r + \mathbf{g}_i^r \odot \mathbf{c}_i^r)\big)$

  end for

end for

Phase 2: Reporting *(selected reports condition the core)*

$\mathcal{S} \leftarrow \mathrm{TopK}(\{\rho_i\}, k)$ ▷ delayed sparse reporting

$\mathbf{q} \leftarrow \sum_{i\in\mathcal{S}} \beta_i \mathbf{W}_{\mathrm{rep}} \mathbf{h}_i^R$

$\mathbf{H}_{\mathrm{core}} \leftarrow F_\theta\big(\mathrm{LayerNorm}(\mathbf{X} + \mathbf{g}_{\mathrm{core}} \odot \mathbf{q})\big)$

return $\mathbf{H}_{\mathrm{core}} \mathbf{W}_{\mathrm{lm}}^\top$ (or pooled task head)

can process. In the second stage, we *unfreeze* the backbone and continue training all parameters jointly. Thus, the trainable parameter count in the joint stage equals the total model parameter count; when Table 5 reports a smaller stage-1 trainable count, it refers to the parameters optimized before backbone unfreezing, including the communication modules and output/task-head parameters when they are not tied to the frozen backbone. For supervision-efficient adaptation, we keep the backbone frozen throughout and train only the non-backbone communication modules, which parallels parameter-efficient fine-tuning but relocates the trainable capacity from adapter or low-rank weights to graph communication parameters.

4.8. Complexity and Implementation Notes

Our default configuration uses N=6 assistants, R=3 deliberation rounds, and k=3 reports. Each round computes $N(N-1)$=30 directed messages, for 90 edge evaluations over three rounds. Each assistant passes through the shared backbone once per round, and the core node once at the end, for $NR + 1 = 19$ shared-backbone applications per forward pass. For comparison, the main LMNet topology (1/4/4/4/1) uses 14 vertex applications and 40 inter-layer edges. LM-LGD therefore requires more computation, but it implements reciprocal multi-round deliberation rather than one-pass propagation. We report parameter counts, training budget, and inference latency alongside accuracy (Section 5). The current implementation also has four scope choices: message passing is hand-written full-neighbor rather than implemented with a general GNN library; top-k is hard selection rather than a differentiable relaxation; the number of rounds is fixed rather than chosen by convergence; and all nodes share one backbone with independent directed edges by default.

4.9. A Walkthrough of the Deliberation Process

We illustrate the forward computation with a multiple-choice example. We first embed the tokens into the hidden states $\mathbf{X}$ and initialize the six assistants from $\mathbf{X}$ with their zero-initialized role embeddings, so every assistant starts from the same pretrained representation of the question. During deliberation, the assistant states gradually separate. In the first round, each assistant reads dense messages from the other five, computes trust weights from its own state and each incoming message, and updates its hidden state through the gate and the shared backbone. For example, one state may encode information about the question setting, while another may track evidence for a competing answer option. Over the second and

third rounds, the assistants exchange and revise these latent interpretations, and the trust weights become more concentrated as the learned roles become more complementary. In the reporting phase, the router assigns relevance scores to the six deliberated states, selects the top three, fuses their projected reports by a softmax, and combines the fused report with the core node through the gated residual. The core node, anchored to the original question representation and conditioned on the fused report, then decodes the final answer. This example follows the same two-phase computation described above. Appendix E provides a symbolic walkthrough on a concrete question.

5. Experimental Setup and Main Results

5.1. Research Questions

We evaluate LM-LGD around three questions.

- **RQ1.** Does latent graph deliberation improve general capability over a single pretrained LM, test-time scaling baselines, and one-pass LMNet-style dense communication?
- **RQ2.** Does LM-LGD support supervision-efficient adaptation when downstream data are limited?
- **RQ3.** Do receiver-conditioned trust aggregation and delayed sparse reporting organize dense communication more effectively than summation, early sparse routing, or dense reporting?

RQ1 and RQ2 follow LMNet's two main evaluation tracks, general capability and limited-supervision adaptation, while RQ3 is specific to LM-LGD's focus on communication organization and is studied through the mechanistic analyses of this section and the ablations of Section 6.

5.2. Experimental Setup

We use Qwen2.5-0.5B and Qwen2.5-1.5B [20] as the shared assistant/core backbone, and additionally Llama3.2-1B, Llama3.2-1B-Instruct [21], GPT2-XL and GPT2-M [22] for adaptation studies. Unless otherwise stated, LM-LGD uses its default configuration (N=6, R=3, k=3, independent directed edges) and the two-stage training schedule of Section 4.7. Training data follow the public mixture used by LMNet: C4 [23], Alpaca [24], ProsocialDialog [25], LaMini-instruction [26], and the training splits of MMLU [27], MATH [28], GSM8K [29], and MBPP [30]; benchmark test sets are excluded from training to avoid leakage. Following the LMNet reference setup, we use only subsets allowed by the same limited-compute regime and keep the benchmark-specific prompting and evaluation conventions aligned with the LMNet protocol. We provide further dataset details and hyperparameters in Appendix C and Appendix A.

We adopt the following conventions throughout. When an experiment reuses a LMNet setting, we keep the same training-data scale, training protocol, and evaluation protocol, and we treat the reported LMNet value as the reference baseline from the original protocol. All LM-LGD numbers are means over three independent runs. Because several reference baselines are available only as mean scores, we do not attach significance claims to single-row differences. If a setting differs from LMNet in data or budget, we note the difference in the corresponding subsection.

5.3. General Capability (RQ1)

We evaluate eleven benchmarks spanning general knowledge, math and science, and coding: MMLU [27], MMLU-Pro [31], BBH [32], ARC-C [33], TruthfulQA [34], GSM8K [29], MATH [28], GPQA [35], MMLU-stem, HumanEval [36], and MBPP [30]. We report accuracy or pass@1 following each benchmark's standard protocol, and compare against Prompt, SFT, Self-Refine [7] (×16), Self-Consistency [2] (×16), and LMNet [13]. Table 1 reports results on both Qwen backbones, and Figure 3 visualizes both the per-benchmark scores and the improvement of LM-LGD over LMNet.

On Qwen2.5-0.5B, LM-LGD improves over LMNet on 10 of 11 benchmarks, and we report gains as absolute score differences rather than relative ratios. For example, on GSM8K we raise accuracy from 50.3% to 54.6%, an absolute gain of 4.3 points. The largest improvements concentrate on math and coding,

Table 1: General capability across eleven benchmarks. Each backbone (Qwen2.5-0.5B and Qwen2.5-1.5B) spans its own block of methods. Best per backbone and row in **bold**. LM-LGD numbers are averaged over three independent runs; "*N/A*" marks settings without a standard self-consistency protocol.

		Qwen2.5-0.5B						**Qwen2.5-1.5B**					
Category	Benchmark	Prompt	SFT	Self-Refine	Self-Cons.	LMNet	LM-LGD	Prompt	SFT	Self-Refine	Self-Cons.	LMNet	LM-LGD
General	MMLU	44.3	44.6	45.2	46.0	53.9	**56.8**	60.9	59.3	63.7	61.8	71.2	**73.5**
	MMLU-Pro	15.7	13.2	16.4	17.1	26.2	**28.5**	28.5	26.9	30.4	29.1	40.3	**42.8**
	BBH	20.3	19.9	24.7	21.6	47.3	**50.1**	45.1	44.5	51.2	48.6	63.8	**67.2**
	ARC-C	35.6	34.6	32.8	36.3	38.0	**39.7**	54.7	52.8	54.3	57.2	67.1	**68.4**
	TruthfulQA	40.2	40.5	38.9	41.3	**47.9**	47.2	46.6	47.2	48.9	46.1	**57.5**	56.9
Math & Science	GSM8K	41.6	41.8	42.1	43.5	50.3	**54.6**	68.5	67.3	72.8	71.4	81.6	**84.3**
	MATH	19.5	12.4	19.8	19.0	38.8	**43.5**	35.0	32.1	38.9	36.5	55.3	**59.7**
	GPQA	24.8	22.0	23.5	25.2	25.6	**27.3**	24.2	23.5	25.1	26.8	32.9	**36.1**
	MMLU-stem	39.8	39.8	40.6	41.4	46.0	**49.2**	54.8	52.9	58.4	56.0	68.5	**70.9**
Coding	HumanEval	30.5	27.6	28.7	*N/A*	39.0	**42.8**	37.2	36.0	41.3	*N/A*	52.4	**55.8**
	MBPP	39.3	35.1	40.5	*N/A*	45.8	**48.9**	60.2	57.6	64.5	*N/A*	73.8	**75.1**

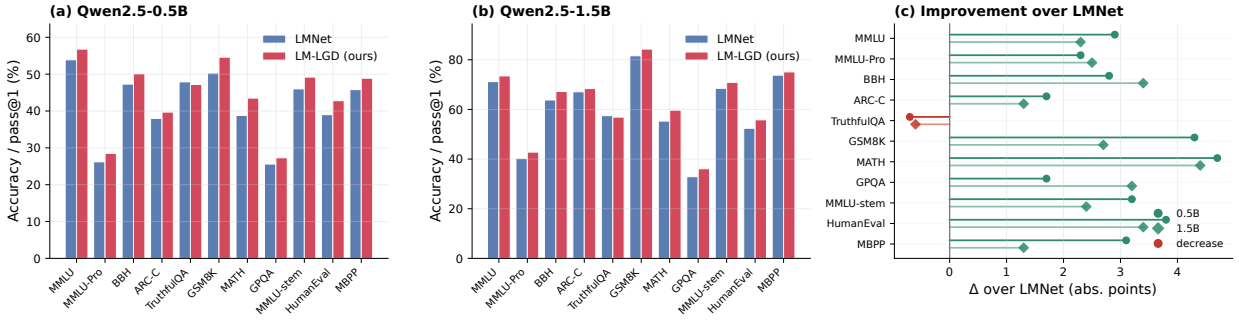


Figure 3: **General capability results.** (a)–(b) Per-benchmark scores for LMNet and LM-LGDon Qwen2.5-0.5B and Qwen2.5-1.5B. (c) Per-benchmark absolute improvement of LM-LGD over LMNet on both backbones; LM-LGD improves on 10/11 benchmarks, with TruthfulQA the single exception.

namely MATH (+4.7 points), GSM8K (+4.3), HumanEval (+3.8), MMLU-stem (+3.2), and MBPP (+3.1), tasks that benefit most from several rounds of latent exploration and refinement before a sparse report is committed. The single exception is TruthfulQA (47.2% vs. 47.9%). On Qwen2.5-1.5B the pattern repeats: LM-LGD again improves on 10/11 benchmarks, with the same TruthfulQA exception and particularly large gains on MATH (+4.4 points), BBH (+3.4), HumanEval (+3.4), and GPQA (+3.2). Because the gains hold across two backbones, we read them as evidence that LM-LGD improves the *organization* of LMNet-style dense communication rather than exploiting a single model. The relative insensitivity of TruthfulQA to latent deliberation highlights a boundary condition: Tasks that depend heavily on calibrated factual recall may not benefit from additional hidden-state exchange, and may even be vulnerable to over-confident signals from sparse reporting.

5.4. Supervision-Efficient Adaptation on Reasoning (RQ2)

We next test whether LM-LGD adapts well under limited supervision, following LMNet's reasoning-adaptation setting. On MMLU we use the full training set (about 100k examples) and report Δ accuracy; on GSM8K we use the same data as LMNet, namely the GSM8K training set (7.47k) plus 93.7k examples from OpenR1-Math-220k, and report final-answer accuracy. We compare against Pred, Prompt, SFT, the

Table 2: MMLU limited-supervision adaptation, Δ accuracy (%). Best in **bold**; "–" denotes an unavailable setting.

Method	Qwen2.5 0.5B	Qwen2.5 1.5B	Llama3.2 1B	Llama3.2 1B-Ins.	GPT2 XL
Pred	22.36	34.75	6.05	20.93	0.24
Prompt	22.50	–	11.69	24.30	1.69
SFT	21.50	35.02	24.65	24.83	7.40
COCONUT	19.22	33.85	9.88	18.54	3.63
CODI	19.48	35.16	20.69	21.30	5.01
LMNet	<u>23.35</u>	<u>37.28</u>	<u>27.19</u>	<u>27.57</u>	<u>13.10</u>
LM-LGD	**27.43**	**39.86**	**28.92**	**28.14**	**14.67**

Table 3: GSM8K limited-supervision adaptation, final-answer accuracy (%). Best in **bold**.

Method	Qwen2.5 0.5B	Qwen2.5 1.5B	Llama3.2 1B	Llama3.2 1B-Ins.
Pred	5.31	9.25	2.88	30.10
Prompt	41.60	68.50	11.49	43.67
SFT w/ CoT	42.94	69.31	38.69	46.39
COCONUT	30.81	48.63	26.79	38.27
CODI	42.06	65.30	37.25	44.58
LMNet	<u>50.82</u>	<u>72.67</u>	<u>44.02</u>	<u>56.15</u>
LM-LGD	**55.74**	**75.28**	**45.83**	**57.52**

latent-reasoning methods COCONUT [11] and CODI [12], and LMNet. Tables 2 and 3 report results across backbones.

LM-LGD improves over LMNet on all five MMLU backbones (an average absolute gain of +2.1 points in Δ-accuracy) and all four GSM8K backbones (an average absolute gain of +2.7 points), and remains the strongest method overall. Gains are largest on the weaker Qwen2.5-0.5B (+4.08 points on MMLU, +4.92 points on GSM8K) and shrink as the backbone becomes stronger or instruction-tuned, which indicates diminishing returns when the base model already leaves little room for communication-based improvement. The comparison with COCONUT and CODI is informative: both also operate in a latent or continuous space, but as chain-like or self-iterative computation. LM-LGD instead organizes multiple assistant states as a graph that explores in parallel, corrects reciprocally, and reports selectively. Its consistent edge over these baselines suggests that supervision-efficient reasoning benefits not only from moving computation into latent space, but from *organizing* that latent communication across nodes.

5.5. Data-Efficient Adaptation on E2E (RQ2)

We further evaluate data-to-text generation on E2E [37] with a GPT2-M backbone, following the PEFT comparison of LMNet and LoRA-style baselines [38, 39, 40]. We report BLEU, NIST, MET, ROUGE-L, CIDEr (higher is better), and an average rank (lower is better). Table 4 shows the results; entries marked * are inherited from the PEFT baselines under the same E2E setting.

Base LM-LGD improves over LMNet on all five metrics and attains a better average rank (3.2 vs. 4.6). Moreover, LM-LGD is complementary to PEFT: LM-LGD+Prefix is best on every metric, and LM-LGD+LoRA also surpasses both LMNetand base LM-LGD. In contrast, LM-LGD+FT degrades sharply, consistent with overfitting when the full backbone is updated on small data, which supports our default of freezing the backbone while training communication modules.

Table 4: E2E data-to-text adaptation with GPT2-M. Higher is better except Rank Avg. Best in **bold**. *: PEFT baseline under the same setting.

Method	BLEU	NIST	MET	ROUGE -L	CIDEr	Rank Avg.
No Adaptation	0.00	0.42	0.04	0.16	0.00	12.0
Full Fine-Tuning*	68.2	8.62	46.2	71.0	2.47	7.5
Adapter$_L$ (0.37M)*	66.3	8.41	45.0	69.8	2.40	10.0
Adapter$_L$ (11.09M)*	68.9	8.71	46.1	71.3	2.47	6.9
Adapter$^*_H$	67.3	8.50	46.0	70.7	2.44	9.0
Prefix*	69.7	8.81	46.1	71.4	2.49	5.6
LoRA*	68.9	8.68	46.5	71.5	2.51	5.1
LMNet	70.5	8.85	46.5	71.5	2.48	4.6
LM-LGD	71.3	8.92	46.6	72.0	2.49	3.2
LM-LGD + Prefix	**72.3**	**8.97**	**46.8**	**72.5**	**2.55**	**1.0**
LM-LGD + LoRA	71.9	8.94	46.6	72.2	2.53	2.1
LM-LGD + FT	65.7	8.38	42.1	67.9	2.01	11.0

Table 5: Efficiency comparison (Qwen2.5-0.5B backbone). Backbone applications and directed edges are counted per forward pass. "Trainable" reports the stage-1 active parameters before backbone unfreezing; after stage-2 joint training, all total parameters are trainable. "Comm." counts only graph communication modules. LM-LGD uses N=6, R=3, k=3.

	Single LM	LMNet 1/4/4/4/1	LM-LGD
Total params (M)	494	612	658
Stage-1 trainable params (M)	494	230	276
Comm. params (M)	0	118	164
Backbone apps / forward	1	14	19
Directed edges / forward	0	40	90
Rel. inference latency	1.0×	6.8×	9.4×

5.6. Efficiency and Budget

Because the Reciprocal LM Graph is heavier than LMNet's one-pass topology, we report cost explicitly in Table 5. LM-LGD adds communication parameters and more backbone/edge applications per forward pass, but keeps the parameter count controlled through backbone sharing: the dominant parameter mass remains one shared backbone, and the communication modules add a modest overhead. Our aim is not to claim superiority at equal FLOPs, but to show that, at a comparable data and parameter budget but with higher compute, multi-round latent deliberation yields consistent accuracy gains. Where latency matters, this trade-off should be weighed against the gains in Tables 1 to 4, and we discuss it as a limitation in Section 7.

5.7. Communication and Mechanistic Analysis (RQ3)

Finally, we examine *how* LM-LGD communicates, which directly supports the deliberation claim. Figure 4 reports three diagnostics on Qwen2.5-0.5B. The receiver-conditioned trust matrix (Figure 4a) is clearly non-uniform: assistants are not used interchangeably, and each receiver concentrates trust on a few senders. The router selection frequency (Figure 4b) shows specialization without collapse: some assistants are selected far more often than others, yet no assistant is entirely ignored. Trust entropy (Figure 4c) decreases monotonically across rounds and stays below the uniform-distribution ceiling, indicating that the model starts by listening broadly and becomes progressively more selective as deliberation proceeds.

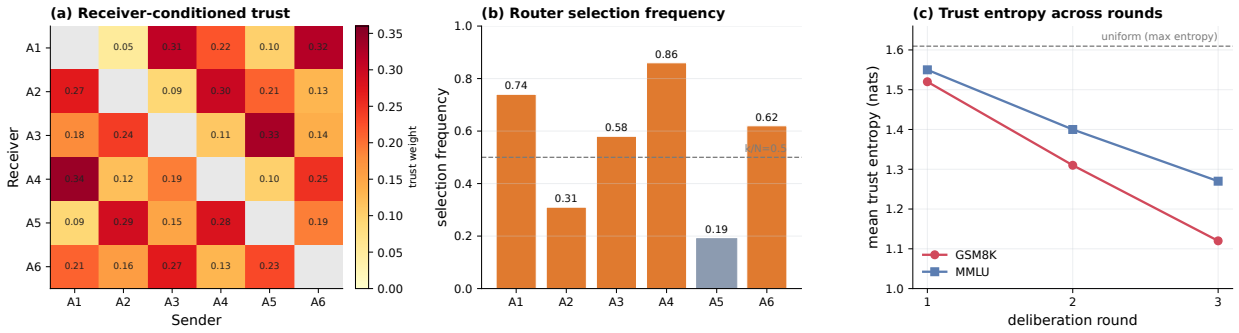


Figure 4: **Mechanistic analysis of LM-LGD** (N=6). (a) Receiver-conditioned trust matrix (rows: receivers, columns: senders; diagonal masked). (b) Router top-k selection frequency per assistant. (c) Mean trust entropy across deliberation rounds, with the uniform-distribution ceiling $\ln 5$ marked. Communication is non-uniform and grows more selective over rounds.

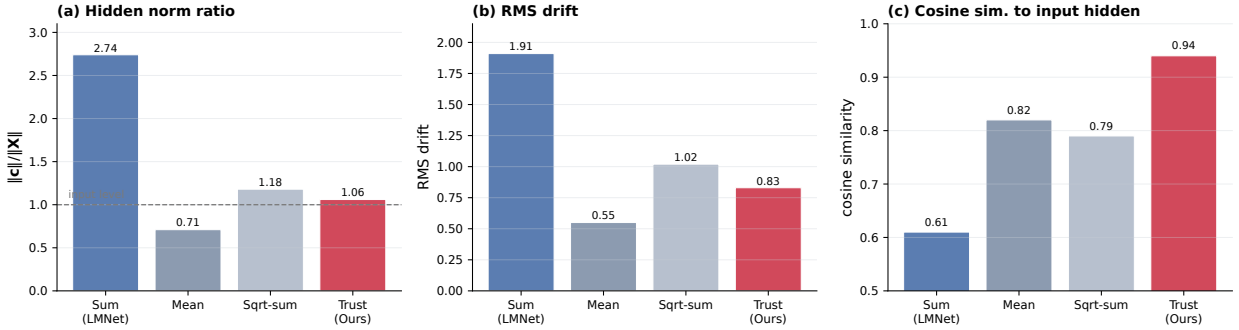


Figure 5: **Hidden-state distribution-shift diagnostics** across aggregation rules. (a) Norm ratio relative to the input hidden state; (b) RMS drift; (c) cosine similarity to the input hidden state. LMNet's unnormalized *Sum* inflates scale and drifts most in direction, while LM-LGD's trust aggregation stays closest to the input distribution.

Figure 5 quantifies hidden-state distribution shift across aggregation rules. The unnormalized sum used by LMNet inflates the hidden norm to 2.7× the input scale and exhibits the largest RMS drift and the lowest cosine similarity to the input hidden state. Mean aggregation and square-root-normalized summation control scale but distort direction, whereas LM-LGD's receiver-conditioned trust aggregation stays closest to the input distribution on all three diagnostics (norm ratio 1.06, cosine 0.94). This corroborates our argument that the issue with summation is not only scale but *usefulness-weighted* aggregation, and that trust-based, gated, normalized integration is a more stable way to organize dense communication. Together, these analyses answer RQ3: LM-LGD learns structured, specialized communication rather than merely adding parameters.

5.8. Case Studies of the Deliberation Process

To make the deliberation concrete, we visualize the per-node attention patterns of a single forward pass on two examples, one from GSM8K and one from MMLU (Figures 6 and 7). For each assistant and round, we take the final-layer, final-head attention matrix from that run and visualize a low-dimensional projection of the aggregated token-position weights. These figures are qualitative snapshots of one forward pass rather than aggregate measurements. For each example, we render the attention map of every assistant at every round, so the figure reads left to right as the two phases of LM-LGD: a deliberation phase in which the six assistants exchange and revise hidden states over three rounds, and a reporting phase in which the router selects three assistants whose deliberated states are fused into the core node before it decodes the answer.

Two qualitative patterns recur. First, the attention maps grow more peaked from round 1 to round 3: early rounds spread attention broadly, whereas later rounds concentrate on a few salient positions, mirroring

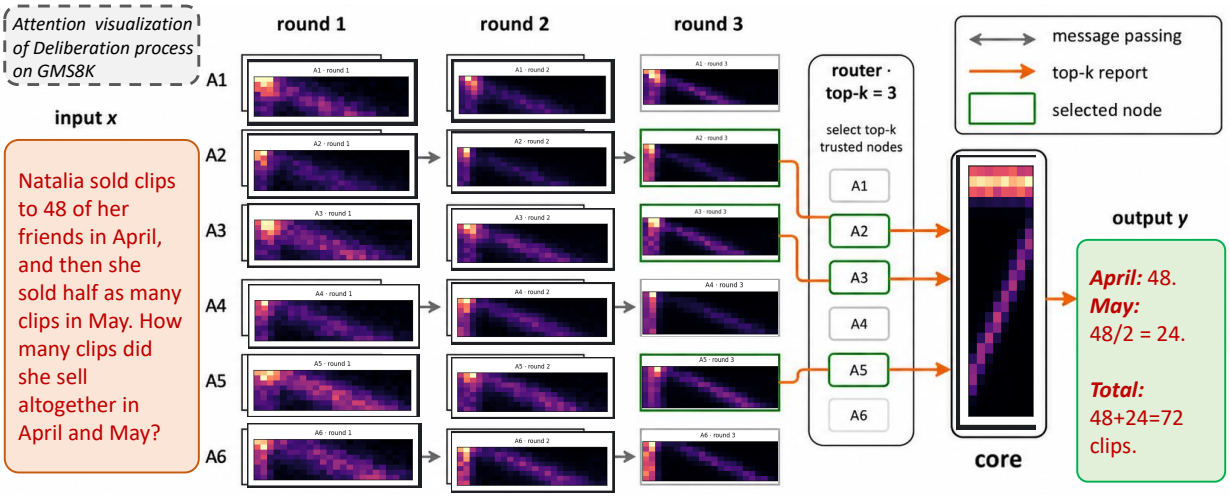


Figure 6: **Deliberation process on a GSM8K example.** From left to right: the input problem; three columns of deliberation rounds, each showing the attention maps of assistants A_1 to A_6; the router that selects the top-k=3 assistants; and the core node that fuses the selected reports and decodes the answer. Attention sharpens across rounds, and different assistants attend to different parts of the problem, illustrating role specialization during deliberation.

the decreasing trust entropy of Figure 4c. Second, the assistants do not converge to a single behavior. Different assistants attend to different spans of the input, and the router selects a different subset on the two examples, which matches the specialization-without-collapse pattern reported above. The core node, by contrast, shows the most focused attention of all, consistent with its role of committing a single decision from the fused reports. These case studies turn the aggregate statistics into a readable picture of how the two phases of LM-LGD operate on individual inputs.

6. Ablation Studies and Mechanistic Analysis

We use the ablation studies to identify which parts of LM-LGD are responsible for the gains observed in Section 5. The method combines several design choices: the width of the assistant graph, the number of deliberation rounds, delayed sparse reporting, role-preserving initialization, trust aggregation, gated updates, and controlled core injection. Each ablation below isolates one design question while keeping the backbone, training data, and budget approximately fixed. Unless noted otherwise, all LM-LGD numbers in this section are obtained on Qwen2.5-0.5B and averaged over three independent runs.

6.1. Number of Assistants

We first examine how the width of the reciprocal graph affects latent deliberation. This experiment varies the number of assistant nodes $N \in \{3, 4, 6, 8\}$ while keeping the default three rounds and top-3 reporting. We use this experiment to test whether additional assistants provide useful diversity or mainly introduce redundant messages and extra computation.

As shown in Figure 8, accuracy improves consistently when moving from three to six assistants: GSM8K rises from 52.4% to 54.6%, MMLU from 55.1% to 56.8%, and MATH from 41.0% to 43.5%. Increasing the graph to eight assistants does not bring further gains and slightly lowers accuracy on all three datasets. This pattern suggests that a small assistant set limits the diversity of deliberation, while an overly wide graph produces additional states that the router cannot reliably turn into useful evidence. We therefore adopt N=6 as the default because it gives the best accuracy–cost trade-off in this setting.

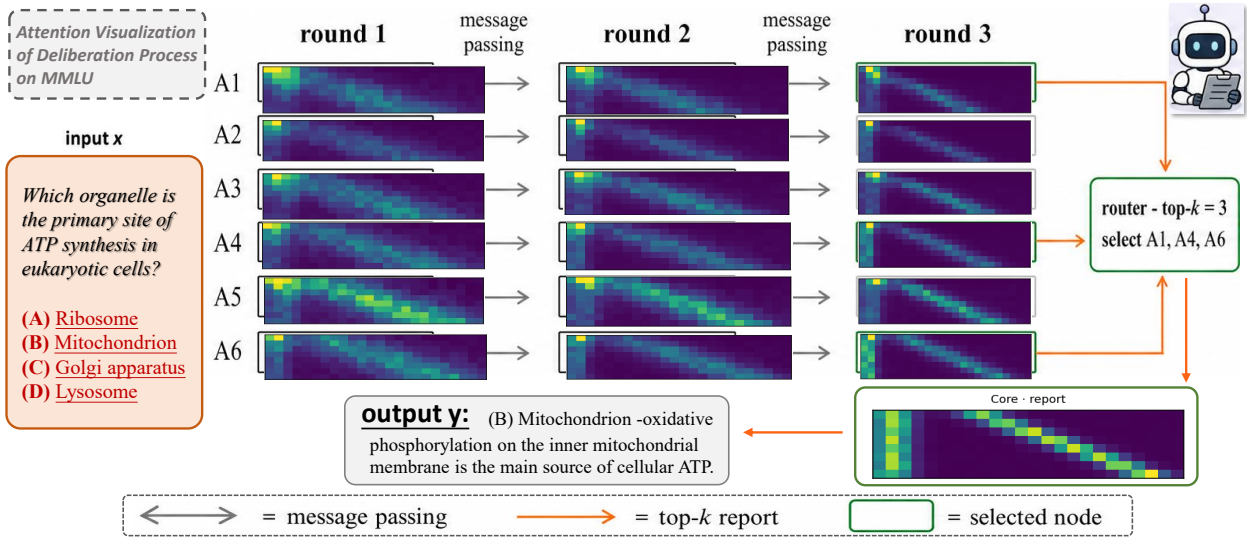


Figure 7: **Deliberation process on an MMLU example.** The same layout as Figure 6 for a multiple-choice question. The selected assistants differ from those chosen for the GSM8K example, which is consistent with the task-dependent routing observed in Figure 4b.

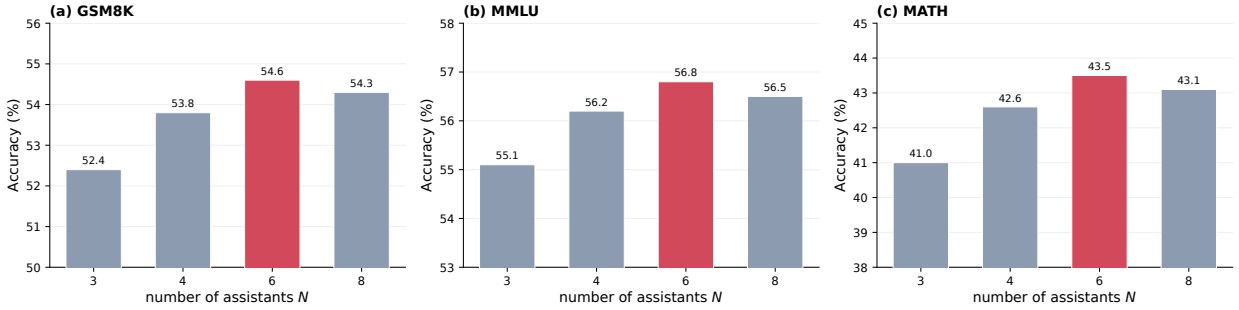


Figure 8: **Effect of the number of assistants** N on (a) GSM8K, (b) MMLU, and (c) MATH (Qwen2.5-0.5B). Accuracy improves up to N=6 (highlighted) and then saturates; more assistants add cost without further gains.

6.2. Number of Deliberation Rounds

We next study how much iterative communication is beneficial after the graph width is fixed. With N=6, we sweep the number of deliberation rounds $R \in \{1, 2, 3, 4, 5\}$ on five datasets and repeat the study on both Qwen2.5-0.5B and Qwen2.5-1.5B. We use this study to determine whether the gains come from a single exchange of messages or from multi-round revision among assistants.

In Figure 9, one round is not enough for the assistants to fully exploit the reciprocal graph. Accuracy improves from R=1 to R=3 on most datasets and on both backbones. After that point, additional rounds often reduce accuracy, which indicates that repeated message passing can over-smooth assistant states and weaken their specialization. The optimal depth also depends on the task: MATH benefits from a fourth round, whereas BBH peaks after two rounds. These results suggest that deliberation depth reflects the structure of the task rather than only the size of the backbone. We use R=3 as the default because it gives the best average behavior across backbones, while the task-specific peaks motivate future adaptive stopping mechanisms.

6.3. Number of Reports (top-k)

We then analyze the sparsity of the final reporting stage. Holding N=6 and R=3 fixed, we vary the number of reported assistants $k \in \{1, \ldots, 6\}$, where k=6 passes information from all assistants to the core

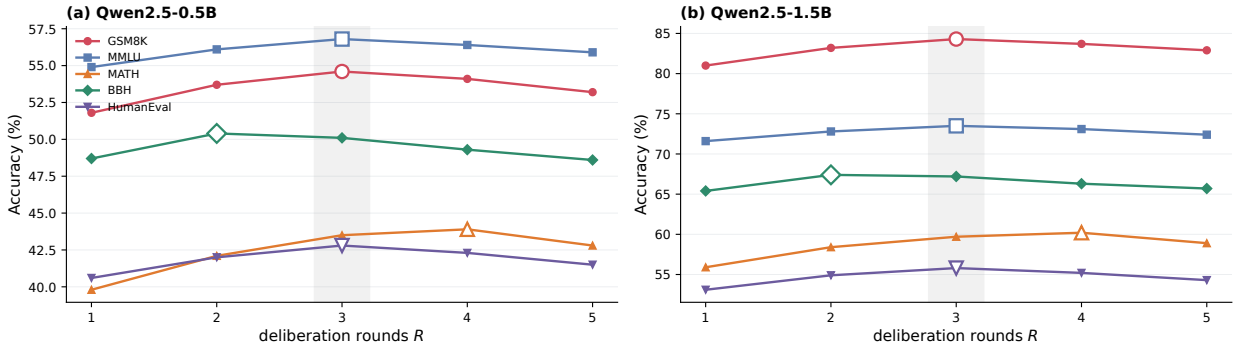


Figure 9: **Effect of deliberation rounds** R on five datasets for (a) Qwen2.5-0.5B and (b) Qwen2.5-1.5B (N=6). Most datasets peak at R=3 (shaded), while MATH peaks at R=4 and BBH at R=2 on both backbones; beyond the peak, extra rounds reduce accuracy as assistant states over-smooth.

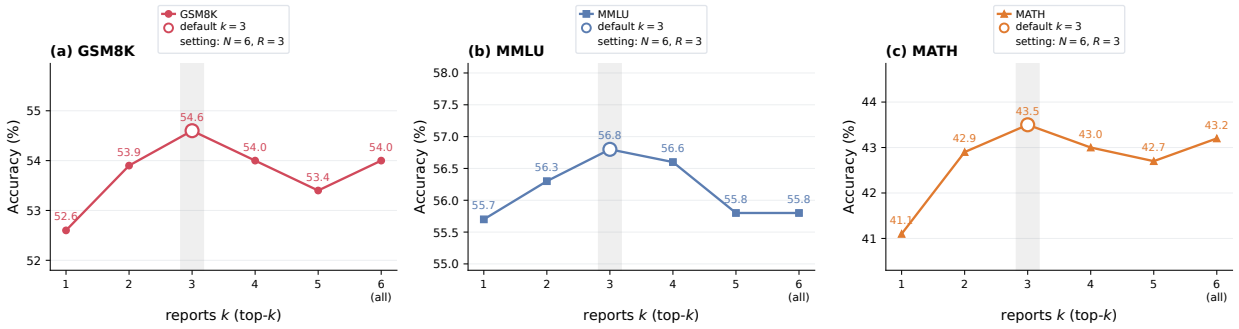


Figure 10: **Effect of the number of reports** k on (a) GSM8K, (b) MMLU, and (c) MATH (Qwen2.5-0.5B, N=6, R=3). Accuracy values are annotated above each point. The highlighted default k=3 gives the best result on all three datasets; k=6 reports all assistants but remains below the default.

node. This ablation asks whether the final decision benefits from all deliberated states or from a compact selected set.

As shown in Figure 10, accuracy peaks at k=3 on all three datasets, reaching 54.6% on GSM8K, 56.8% on MMLU, and 43.5% on MATH. With only one report, the core node receives too little complementary evidence. Increasing k beyond three also does not improve the final decision: reporting all six assistants reaches 54.0%, 55.8%, and 43.2% on the three datasets, respectively. The result supports the design of delayed sparse reporting: assistants should deliberate densely, but the core node should receive a selected set of reports at decision time.

6.4. Role-Identity Initialization

We further examine how assistants should acquire distinct roles. The design challenge is that the role embedding must break symmetry among assistants without moving their initial hidden states away from the pretrained distribution. To isolate this effect, we compare five initialization strategies on three datasets: no identity at all, random identities, small-magnitude random identities, a learnable scaled identity $\mathbf{X} + \gamma\,\mathbf{p}_i$ with γ learned from zero, and our default zero-initialized additive identity.

Figure 11 shows a clear separation between distribution-aware and distribution-disruptive initialization. Random identities are consistently the worst, even below using no identity, because they perturb the shared backbone before the communication modules have learned how to use role information. Strategies that start at or near the pretrained distribution do better, and zero initialization gives the best result on GSM8K, MMLU, and MATH. This supports the role-preserving design in Section 4.1: assistant specialization should be learned gradually from a stable hidden state, rather than imposed through random offsets at initialization.

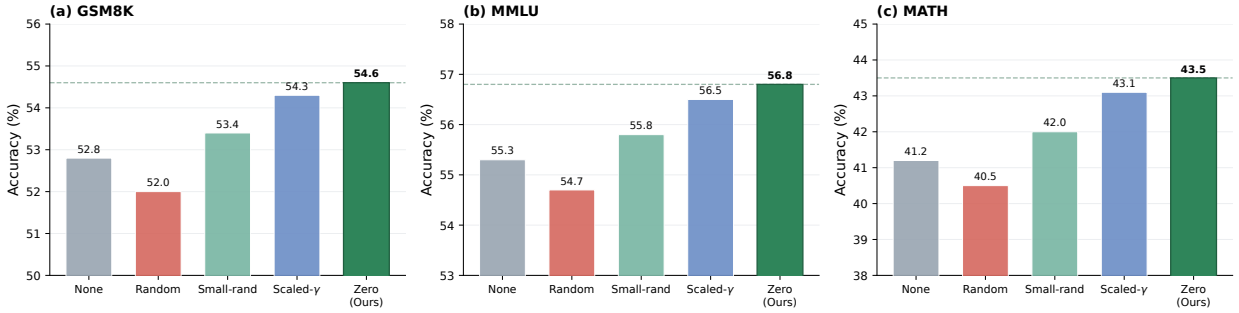


Figure 11: **Role-identity initialization** on (a) GSM8K, (b) MMLU, and (c) MATH (Qwen2.5-0.5B). Zero-initialized additive identities (highlighted) are best; random identities are worst, even below using no identity, illustrating the cost of perturbing the pretrained hidden distribution at initialization.

6.5. Joint Effect of Graph Width and Reporting Sparsity

The previous two ablations vary N and k separately. We further study their joint effect because the two choices are coupled: increasing the number of assistants expands the pool of deliberated states, but the router still needs to decide how many of those states should reach the core node. To analyze this interaction, we sweep valid pairs (N, k) with $N \in \{3, 4, 6, 8\}$ and $1 \leq k \leq N$, fixing R=3 and using the same Qwen2.5-0.5B setting.

Figure 12 shows that the best region is concentrated around a moderately wide graph and a compact reporting set, rather than at either extreme. MMLU and MATH peak at the default pair (N=6, k=3), with 56.8% and 43.5%, respectively. GSM8K and HumanEval obtain their highest values at (N=8, k=4), reaching 54.8% and 52.8%, but their surfaces still follow the same qualitative pattern: very small k underuses the assistant pool, while reporting nearly all assistants does not reliably improve the decision. The joint sweep therefore refines the interpretation of the one-dimensional ablations. The default (N=6, k=3) is a strong and stable operating point across tasks, whereas slightly wider graphs can help some tasks when the reporting budget is increased only modestly. This suggests that future versions of the router could choose k adaptively as a function of both the input and the available assistant pool.

6.6. Reporting, Aggregation, and Message Passing

We next test whether the main gains require the specific mechanisms in LM-LGD or whether simpler substitutions can recover the same behavior. We replace three core components one at a time on GSM8K: the reporting strategy, the aggregation rule, and the latent-state update. Full LM-LGD reaches 54.6% in this setting.

Figure 13(a) evaluates the reporting mechanism. Dense reporting (52.9%), early per-round top-k (52.1%), and random top-k (51.4%) all underperform delayed top-k (54.6%). The comparison separates two effects: sparsity is useful only after assistants have finished deliberating, and the selected reports must be chosen by the learned router. Figure 13(b) evaluates aggregation. Replacing trust with summation (51.0%), mean aggregation (52.4%), square-root-normalized summation (52.9%), or a uniform softmax (52.6%) consistently reduces accuracy. Normalization helps relative to raw summation, but the remaining gap to trust aggregation shows that input-dependent weighting is necessary, consistent with the distribution-shift diagnostics in Figure 5. Finally, Figure 13(c) evaluates the message-passing update. A GRU-style update is the strongest alternative (54.1%), but it still trails the gated trust update, while hard max-trust selection is weakest (52.0%) because it forces each receiver to rely on a single sender. Together, these ablations show that LM-LGD benefits from the combination of delayed reporting, receiver-conditioned trust, and gated multi-message integration.

6.7. Backbone, Edge, and Core-Injection Variants

Finally, we examine several implementation choices that affect how the latent graph is trained and how its messages enter the shared backbone. These ablations are reported as numerical tables in Appendix B;

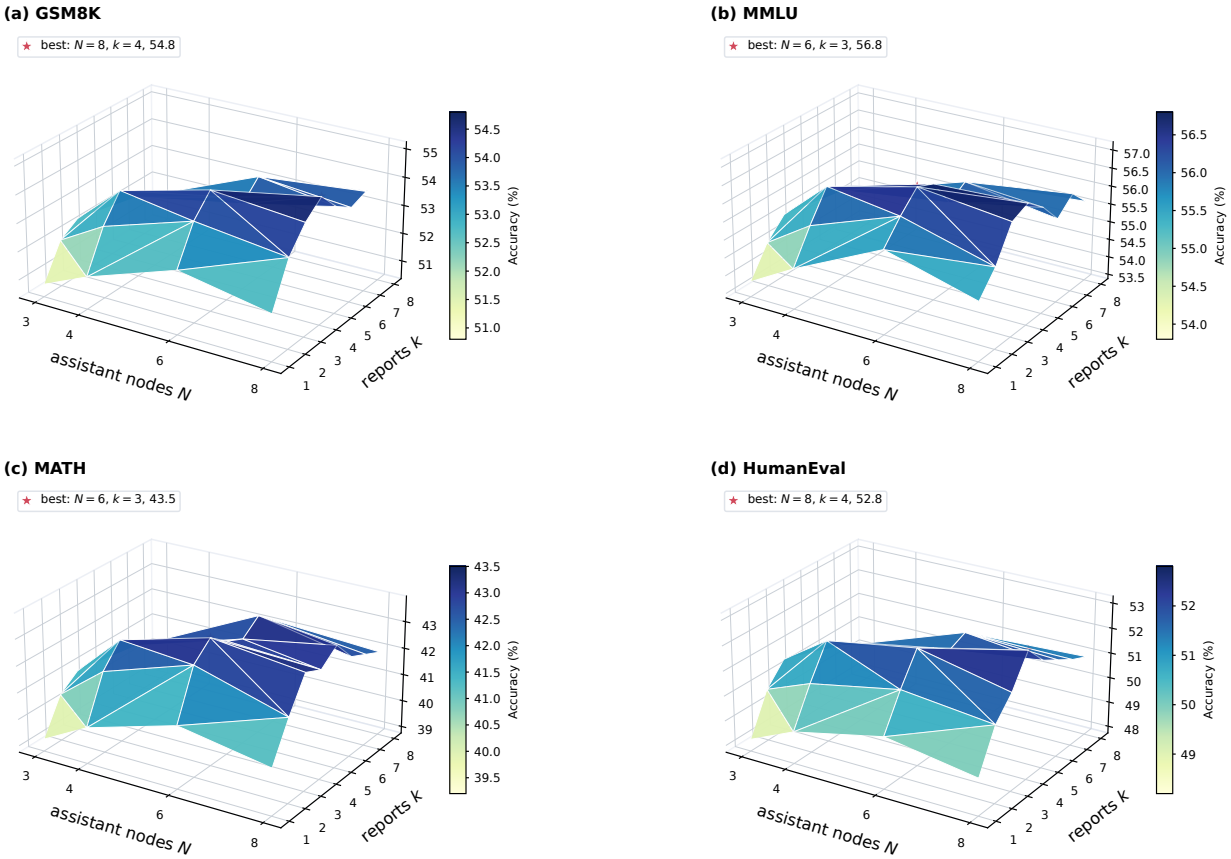


Figure 12: **Joint effect of graph width and reporting sparsity.** We vary the number of assistant nodes N and the number of reported assistants k on (a) GSM8K, (b) MMLU, (c) MATH, and (d) HumanEval (Qwen2.5-0.5B, R=3). Each surface is drawn over valid pairs $k \leq N$, with color indicating accuracy. The marked point denotes the best pair in each panel. The surfaces show that performance is highest around compact reporting from a moderately wide graph, rather than at the sparsest or densest reporting extreme.

they use GSM8K accuracy on Qwen2.5-0.5B with full LM-LGD at 54.6%.

The training-schedule ablation asks whether communication modules should be trained together with the backbone from the beginning. Training all parameters from scratch reaches 53.1%, while the two-stage schedule reaches 54.6%. Keeping the backbone frozen throughout is competitive at 53.9%, which indicates that a substantial part of the gain comes from learning the graph communication itself. For edge parameterization, independent directed edges reach 54.6%, compared with 53.4% for a shared edge, 53.2% when edge identities are removed, and 52.7% for a token-wise MLP edge. These results indicate that direction-specific sequence transformations help assistants form specialized communication channels. For core injection, gated addition with normalization is best at 54.6%; direct addition drops to 52.5%, and removing the report projection gives 52.9%. The core node therefore needs a controlled residual interface rather than a raw addition of assistant states.

6.8. Summary of Findings

Overall, these above ablations lead to three major conclusions: First, the graph should be wide enough and deep enough to support useful cross-assistant revision, but both width and depth saturate; more nodes or more rounds do not automatically help. Second, trust-guided aggregation is not only a normalization device. It gives each receiver an input-dependent way to weight incoming messages, which improves accuracy

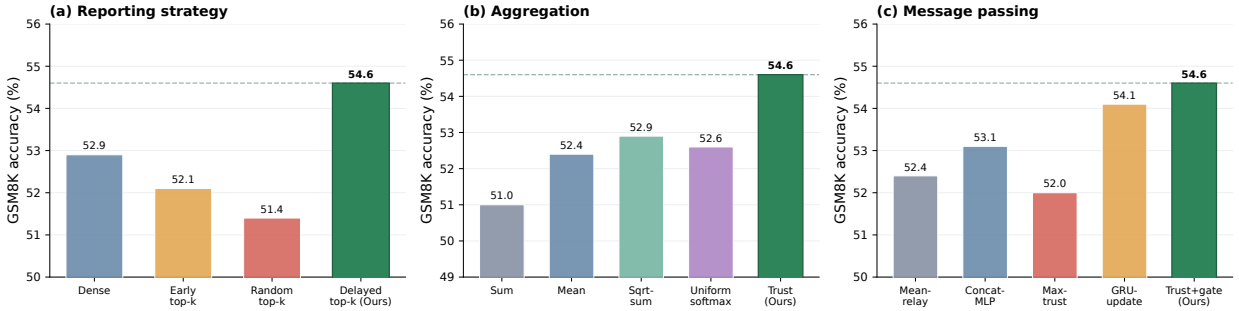


Figure 13: **Mechanism ablations** on Qwen2.5-0.5B / GSM8K. (a) Reporting strategy, (b) message aggregation, and (c) message-passing update. The dashed line marks full LM-LGD; the highlighted bar is our default. Each design choice contributes to the final result.

and reduces hidden-state drift relative to raw summation, mean aggregation, and square-root-normalized summation. Third, sparsity is most useful at the reporting stage rather than during deliberation. In combination with the mechanistic analyses in Section 5.7, these results support the central design principle of LM-LGD: language-model nodes should exchange information densely while they deliberate, and commit only selected, trusted information to the final decision.

7. Discussion

7.1. From a Communication Medium to a Communication Organization

LMNet [13] introduced dense hidden-state communication between language-model nodes, avoiding the discretization and information loss caused by natural-language messages in multi-agent debate and workflow systems [3, 9]. We view this as an important step, but the communication medium itself only specifies how nodes exchange information. It does not specify which nodes should exchange information, which messages should receive higher weight, or when the system should stop exploration and produce an answer. Our work addresses these organizational questions. We keep the dense channel of LMNet and study how the information flow over that channel should be structured.

We describe our answer in two stages. During deliberation, every assistant receives information from every other assistant, so information can circulate freely and each node can revise its state using the messages produced by its peers. At decision time, we use a narrower reporting channel, so the core node receives a small set of trusted deliberated states rather than an unnormalized sum of all messages. From this perspective, LMNet and LM-LGD address two aspects of the same problem: LMNet makes dense communication possible, and LM-LGD organizes that communication with reciprocity, trust, and a clear separation between deliberation and decision. Table 1 shows the empirical effect of this organization: using the same dense medium, LM-LGD improves results on nearly every benchmark and on two different backbones.

7.2. Why Two Phases, and Why Reporting Comes Last

We designed the deliberation and reporting phases to manage a trade-off between broad exploration and final selection. Early pruning may remove assistants whose contributions are not yet apparent, preventing their information from reaching the final decision. Propagating all assistants to the core node has the opposite problem: the decision is based on a crowded and partly redundant signal, and our diagnostics show that this signal moves the core node away from the hidden-state distribution seen by the pretrained backbone. We use delayed sparse reporting to separate these two requirements. The assistants first explore with a wide, full-neighbor channel for several rounds, and the router selects a small, complementary set only after deliberation has produced updated states.

Our ablations support this design choice. The top-k ablation (Section 6.3) and the mechanism ablation (Section 6.6) show that early pruning underperforms no pruning, random selection performs worse, and

aggregating all assistants is less effective than aggregating a selected subset. These results explain why we do not name the whole method after delayed sparse reporting. Reporting is the last step of a larger process; the gains depend on multi-round reciprocal exchange and trust-weighted aggregation as well as on the sparsity of the final report.

7.3. How LM-LGD Differs from LMNet-Style Architectures

We summarize the key differences between LMNet-style architectures and LM-LGD, since the two share a backbone and a dense communication medium. The first difference is the graph structure. LMNet arranges nodes in layers and sends information forward once, so a node in a later layer can read earlier nodes but cannot send information back to them. In contrast, our assistants sit in a single reciprocal graph and exchange messages in both directions over several rounds. This lets a node revise an earlier state using later messages, in the spirit of message passing in graph networks [15].

The second difference is aggregation. LMNet adds incoming messages without weights, so every sender contributes equally and the summed signal grows with the number of senders. We use a trust-weighted average whose weights depend on the receiver state, the candidate message, the input, and the node identities. This lets a node rely more on senders that are useful for the current input while keeping the aggregate scale controlled. The third difference is the decision interface. LMNet derives the final answer from the last layer of the same forward graph, whereas we use a dedicated core node that does not participate in the deliberation rounds and receives only a selected, fused report through a gated residual. This keeps the core node close to the pretrained distribution while conditioning it on selected deliberated states.

These differences lead to distinct system behavior. They help keep the shared backbone close to its native input distribution, as Figure 5 shows, and our ablations isolate them as sources of the accuracy gains. Compared to stacking more LMNet layers, which would deepen a one-pass computation and increase aggregation drift, our design changes the computation from feed-forward propagation to reciprocal deliberation, at the cost reported in Section 5.6.

7.4. Distribution Shift in Dense Language-Model Graphs

A central challenge in dense language-model graphs is hidden-state distribution shift. A pretrained transformer expects inputs that resemble its embedding outputs, yet dense graphs feed intermediate nodes with transformed and combined hidden states. When several nodes share one backbone, the same backbone must process inputs from many positions in the graph. We treat this as a core design constraint. Zero-initialized role embeddings start every assistant from the native distribution, softmax trust turns aggregation into a convex combination instead of an unbounded sum, gated residual updates and gated core injection admit external information under learned control, layer normalization limits scale drift, and delayed reporting keeps sparsification from disturbing exploration.

Each component is standard, but together they address a problem left open by LMNet-style summation. The diagnostics in Figure 5 and the initialization ablation in Section 6.4 show that these design choices are not cosmetic: they reduce the shift seen by the shared backbone and contribute to the performance gains. This interpretation also connects LM-LGD to deliberation networks, which revise a decoded draft with a second pass [14], and to latent-reasoning methods that iterate within a single model [11, 12]. LM-LGD differs from both by keeping the iteration entirely in hidden space and distributing it across several communicating nodes rather than a single model. We interpret the consistent exception on TruthfulQA [34] in the same way: tasks that hinge on calibrated factual recall appear to benefit less from extra latent deliberation, and sparse reporting can occasionally amplify a confident but incorrect assistant. We explicitly report this result because it identifies a boundary condition for latent deliberation.

7.5. Limitations

LM-LGD improves performance across our benchmarks, but it has several limitations. The reciprocal graph is more expensive than a one-pass network: our default configuration evaluates ninety directed edges and nineteen shared-backbone applications per forward pass, so inference is slower than LMNet and slower than single-model prompting, even though backbone sharing keeps the parameter count modest. The method

also requires white-box access to hidden states and backbone weights, which precludes use of closed API-based models. Its messages are dense vectors and are therefore harder to inspect than natural-language exchanges. Trust matrices, router statistics, and the case studies in Section 5.8 provide partial visibility, but they do not make the communication fully transparent.

On the modeling side, the top-k selection is a hard, non-differentiable step, and only the fusion over the selected reports is learned by gradients. The number of deliberation rounds is also fixed in advance rather than decided per input, and the current implementation passes messages with hand-written full-neighbor loops rather than a general graph library. We explicitly report these limitations because they define the current operating regime of LM-LGD: learnable dense collaboration among language models with internal access. They also suggest directions for future work.

8. Conclusion and Future Work

Effective language-model collaboration requires more than a channel for passing messages. Once dense hidden states can be exchanged between language model nodes, the central challenge is how to organize this communication so that nodes can revise one another, assign input-dependent trust, and provide only useful information to the final decision. We addressed this challenge with LM-LGD, an LM-based Latent Graph Deliberation Network that turns one-pass dense communication into multi-round latent deliberation over a Reciprocal LM Graph. Specifically, we designed receiver-conditioned trust aggregation to replace unnormalized summation with receiver-conditioned trust over incoming messages. We used a gated latent-state update and zero-initialized role embeddings to support reciprocal specialization while keeping the shared backbone close to its native hidden-state distribution. We further introduced delayed sparse reporting and gated core injection so that assistants can exchange information densely during deliberation, while the core node receives only a selected set of deliberated states at decision time. Across two backbones, LM-LGD improves over LMNet across multiple general-knowledge, reasoning, math, science, and coding benchmarks, and also improves supervision-efficient adaptation on MMLU, GSM8K, and E2E. Mechanistic analyses further show that LM-LGD learns structured, specialized communication and reduces hidden-state distribution shift. These results suggest that dense LM communication should be treated not only as a medium, but also as a computational structure. The value of collaboration comes from how latent messages are revised, trusted, and selectively committed before the final prediction. This perspective illustrated in this work shifts the focus from enabling LM nodes to exchange hidden states toward designing the latent deliberation process that determines how those states influence the decision.

In the future, we plan to explore several directions. First, we will improve graph efficiency by replacing the current dense graph with sparse or input-dependent topologies, shared or low-rank edge modules, and per-input activation of only the most useful edges. Second, we will improve routing by replacing the discrete top-k reporter selection with differentiable or Gumbel-based selection, adding load balancing, and making the routing decision task-conditioned and interpretable. Third, we will study adaptive deliberation, where the model decides when to stop based on changes in hidden states, trust entropy, or uncertainty estimates, moving from a fixed number of rounds to per-input computation. Finally, we will incorporate tool, memory, and verifier nodes into the latent graph, allowing deliberation to span models, tools, and feedback within one learnable computational network.

Data and Code Availability

The code implementation of LM-LGD and corresponding adopted datasets are publicly available in `https://anonymous.4open.science/r/LM-LGD-C11D/`.

Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Implementation and Hyperparameters

Architecture configuration. Table A.6 lists the default LM-LGD configuration used in all main experiments unless stated otherwise. The trust scorer and router are two-layer MLPs; the assistant gate, core gate, report projection, and both LayerNorm modules follow Section 4. Edge modules are LMNet-style sequence-to-sequence transformer blocks, instantiated independently per ordered assistant pair (independent directed edges) and shared across rounds. The shared backbone is a single stripped transformer reused by all assistant and core applications.

Table A.6: Default LM-LGD architecture hyperparameters.

Hyperparameter	Value
Number of assistants N	6
Deliberation rounds R	3
Reports (top-k)	3
Edge parameterization	independent directed
Edge sharing across rounds	yes
Shared backbone	yes
Node identity embedding	enabled, zero-init
Trust scorer hidden size	256
Router hidden size	256
Pooling for scoring	last valid token
Gate activation	sigmoid
Normalization	LayerNorm (ϵ = backbone RMS eps)
Router load-balance weight	0.0

Optimization. Table A.7 summarizes the optimization settings. The two-stage schedule first freezes the backbone and trains only the communication modules (stage 1), then unfreezes all parameters (stage 2), matching the LMNet protocol. For supervision-efficient adaptation we optionally keep the backbone frozen throughout. We use AdamW with cosine decay and a linear warmup, mixed-precision training, and gradient clipping. Exact token budgets follow the LMNet reference settings for each reused experiment; the general-capability setup follows the same limited-compute regime reported by LMNet (less than 200 NVIDIA A100 GPU-days and less than 0.1T training tokens in the reference run).

Table A.7: Optimization settings. Ranges indicate values swept per backbone / task; defaults are typical values.

Setting	Value
Optimizer	AdamW (β_1=0.9, β_2=0.95)
Weight decay	0.1
Peak learning rate (stage 1)	3e-4
Peak learning rate (stage 2)	5e-5
LR schedule	cosine decay, linear warmup
Warmup ratio	0.03
Gradient clipping	1.0
Precision	bf16 mixed precision
Max sequence length	1024
Batch size (tokens)	$\sim$256k
Random seeds	3 independent runs

Parameter groups. The stage-1 trainable parameters fall into communication modules (edge modules, trust scorer, router, node/edge identity embeddings, fusion gates, normalization, and report projection) plus the output or task head when it is not tied to the frozen backbone. In stage 2 the shared backbone joins these modules, so all parameters in the "total parameters" row of Table 5 are trainable. The "communication parameters" row in Table 5 isolates only the graph communication modules.

Appendix B. Additional Ablation Tables

This appendix provides the numerical tables behind Section 6.7. All values are GSM8K final-answer accuracy (%) on Qwen2.5-0.5B, averaged over three runs, with full LM-LGD at 54.6.

Table B.8: Backbone training and edge parameterization ablation.

Variant	GSM8K
Train all from scratch	53.1
Frozen backbone (comm. only)	53.9
Two-stage (default)	**54.6**
Shared edge module	53.4
Independent directed edges (default)	**54.6**
Edge identities off	53.2
MLP edge (vs. transformer block)	52.7

Table B.9: Gated core report-injection ablation.

Injection variant	GSM8K
Direct addition	52.5
Addition + LayerNorm	53.3
Gated addition	53.8
Gated addition + LayerNorm (default)	**54.6**
Concatenation + projection	53.6
No report projection	52.9

These tables reinforce the conclusions of Section 6: two-stage training is the most stable, independent directed edges and a transformer-block edge module outperform their lighter counterparts, and a gated, normalized core injection with a report projection is the most reliable way to admit deliberated information into the core node.

Appendix C. Datasets

Our training and evaluation data follow the public mixture used by LMNet [13]. As in that work, only a subset of these datasets is consumed under our limited compute budget, and benchmark test splits are never used for training. We also keep the same benchmark-specific evaluation conventions as LMNet: MMLU uses 5-shot prompting, MMLU-Pro uses 5-shot chain-of-thought prompting, BBH uses 3-shot chain-of-thought prompting, GSM8K uses 4/8-shot chain-of-thought prompting, MATH uses 0/4-shot chain-of-thought prompting, GPQA uses 5-shot chain-of-thought prompting, and HumanEval and MBPP use 0-shot code-generation evaluation. Below we describe each dataset; all statistics refer to the official releases.

Colossal Clean Crawled Corpus (C4)[1] is a large-scale English web corpus derived from Common Crawl through aggressive cleaning, such as removing boilerplate, deduplicating lines, and filtering non-English and low-quality text, introduced alongside the T5 model [23]. It contains on the order of hundreds of billions of tokens and has become a standard source of general-purpose pretraining and language-modeling data. In our setup, C4 supplies broad, domain-diverse text that anchors the shared backbone's language distribution while the communication modules learn to exchange dense messages. We use only the cleaned English split, and only a small fraction of it, consistent with the compute budget inherited from the LMNet reference configuration.

Stanford Alpaca[2] is an instruction-following dataset of 52k instruction–response pairs generated in the self-instruct style from a strong teacher model [24]. Each example pairs a natural-language instruction, an optional input, and a target response, covering open-ended generation, classification, rewriting, and simple reasoning. Alpaca popularized lightweight instruction tuning of small models and is widely reused as a compact, diverse instruction corpus. In our mixture, Alpaca provides instruction-style supervision that encourages the core node to produce helpful, well-formed responses, complementing the broad but unstructured text from C4. We use the dataset as released, without additional filtering, for the instruction-tuning portions of training.

ProsocialDialog[3] is a large-scale multi-turn dialogue dataset designed to teach conversational agents to respond to problematic or sensitive content in a prosocial manner [25]. Built through a human–AI collaborative procedure, it pairs dialogue contexts with responses grounded in social norms, together with safety labels and rules-of-thumb. It contains roughly 58k dialogues spanning tens of thousands of distinct norms. In our setup, ProsocialDialog contributes dialogue-style and safety-aware supervision, broadening the behavioral coverage of the training mixture beyond single-turn instructions. We use the dataset as released and treat its responses as generation targets, without using its auxiliary safety labels as explicit supervision.

LaMini-instruction[4] is a large instruction dataset of roughly 2.58 million examples, distilled from a strong teacher model using both example-guided and topic-guided generation to maximize diversity [26]. It was created to train the LaMini-LM family of compact models and spans a wide range of tasks and phrasings. In our mixture, LaMini-instruction supplies high-volume, diverse instruction–response supervision that helps the small shared backbone generalize across task formulations. Because of our compute budget, we sample only a subset of the full corpus. We use the released instruction–response pairs directly as targets for the instruction-tuning portion of training.

Massive Multitask Language Understanding (MMLU)[5] is a multiple-choice benchmark covering 57 subjects across the humanities, social sciences, STEM, and professional domains [27]. Each question has four options and a single correct answer, and accuracy is the standard metric. MMLU also provides an auxiliary training split, which we use for supervised adaptation; the test split is reserved strictly for evaluation. We additionally report on the STEM subset (MMLU-stem). In our experiments MMLU serves both as a general-capability benchmark (Table 1) and as a limited-supervision adaptation target (Table 2), making it central to both RQ1 and RQ2.

MATH[6] is a dataset of 12.5k competition mathematics problems, each accompanied by a full step-by-step solution and a final boxed answer [28]. Problems span algebra, geometry, number theory, counting and probability, and precalculus, and are labeled with difficulty levels. Evaluation typically measures final-answer accuracy. We use only the training split as part of our reasoning supervision mixture and evaluate on the held-out test split. MATH is a demanding probe of multi-step symbolic reasoning, and in our results it is one of the benchmarks where LM-LGD's multi-round latent deliberation yields the largest gains over one-pass LMNet.

[1]The official release is hosted by the TensorFlow Datasets and Hugging Face projects at `https://huggingface.co/datasets/allenai/c4`.

[2]The official release is available at `https://github.com/tatsu-lab/stanford_alpaca`.

[3]The official release is available at `https://huggingface.co/datasets/allenai/prosocial-dialog`.

[4]The official release is available at `https://huggingface.co/datasets/MBZUAI/LaMini-instruction`.

[5]The official release is available at `https://huggingface.co/datasets/cais/mmlu`.

[6]The official release is available at `https://github.com/hendrycks/math`.

Grade School Math 8K (GSM8K)[7] is a dataset of 8.5k high-quality grade-school math word problems requiring multi-step arithmetic reasoning, split into 7.47k training and 1.32k test examples [29]. Each problem includes a natural-language solution ending in a numeric answer, and final-answer accuracy is the standard metric. Following LMNet, our GSM8K adaptation augments the training split with additional mathematics data from OpenR1-Math-220k to match the reference budget. GSM8K is our primary testbed for supervision-efficient reasoning (Table 3) and for the ablation studies of Section 6, as its multi-step structure cleanly exposes the benefit of reciprocal deliberation.

Mostly Basic Python Problems (MBPP)[8] is a benchmark of about 974 short, entry-level Python programming tasks, each specified by a natural-language prompt, a reference solution, and a few automated test cases [30]. Performance is measured by functional correctness via pass@k, typically pass@1. We use the training split as part of our coding supervision and evaluate on the held-out split. MBPP, together with HumanEval [36], lets us assess whether latent graph LM transfers to program synthesis; in our experiments LM-LGD consistently improves pass@1 over LMNet on both backbones, suggesting that reciprocal hidden-state refinement also benefits structured code generation.

Appendix D. Running-Time Analysis

This appendix analyzes the running time of LM-LGD, first in theory and then with measured latencies. The takeaway is that LM-LGD spends more arithmetic than a one-pass network but, because all nodes share one backbone and the within-round computations are independent, its wall-clock time stays close to the depth of the graph rather than to the total number of node applications. In other words, LM-LGD trades extra computation for accuracy while keeping wall-clock cost manageable.

Appendix D.1. Theoretical Cost

We use four symbols for the dominant operations. Let T_{fwd} be the time of one full shared-backbone pass over a sequence, including decoding, which is by far the most expensive operation. Let T_{edge} be the time of one directed edge module that maps a sender hidden sequence to a receiver-specific message, let T_{agg} be the time to score and softmax-aggregate the incoming messages of one receiver in one round, and let T_{rep} be the one-time cost of the router, the top-k selection, the report fusion, and the gated core injection. The edge, aggregation, and reporting modules are lightweight relative to the backbone, so $T_{\text{edge}}, T_{\text{agg}}, T_{\text{rep}} \ll T_{\text{fwd}}$.

Counting every operation serially, a LM-LGD forward pass with N assistants and R rounds performs $NR+1$ backbone applications, $N(N-1)R$ edge applications, and NR aggregations, plus one reporting step:

$$T_{\text{serial}} = (NR+1)\,T_{\text{fwd}} + N(N-1)R\,T_{\text{edge}} + NR\,T_{\text{agg}} + T_{\text{rep}}. \tag{D.1}$$

For the default N=6, R=3 this is $19\,T_{\text{fwd}}$ and $90\,T_{\text{edge}}$, which is the arithmetic cost reported in Section 5.6. The serial count, however, overstates the wall-clock time, because within a round the N assistant backbone passes are independent and share parameters, so they batch into a single backbone call along an enlarged batch dimension. The $N(N-1)$ edge messages of a round batch the same way. Writing $\hat{T}_{\text{fwd}}$ and $\hat{T}_{\text{edge}}$ for the batched costs, which on parallel hardware grow far more slowly than their counts, the wall-clock time is

$$T_{\text{wall}} \approx (R+1)\,\hat{T}_{\text{fwd}} + R\,\hat{T}_{\text{edge}} + T_{\text{rep}} \;\approx\; (R+1)\,\hat{T}_{\text{fwd}}, \tag{D.2}$$

where the last step uses $\hat{T}_{\text{edge}} \ll \hat{T}_{\text{fwd}}$. The wall-clock therefore scales with the number of rounds R, not with the product NR. For comparison, LMNet with the 1/4/4/4/1 topology has five layers and batches the four nodes of a layer together, so its wall-clock is about $5\,\hat{T}_{\text{fwd}}$, while LM-LGD with R=3 is about $4\,\hat{T}_{\text{fwd}}$. The two are of the same order even though LM-LGD performs more total work, which is the precise sense in which we trade computation for time: the additional assistants and edges raise the arithmetic but not the depth, and the depth is what the wall-clock follows.

[7] The official release is available at `https://github.com/openai/grade-school-math`.

[8] The official release is available at `https://github.com/google-research/google-research/tree/master/mbpp`.

Appendix D.2. Measured Latency

To check the analysis, we measured per-example latency on Qwen2.5-0.5B for ten randomly drawn examples from each of four benchmarks and averaged them. We compare LM-LGD against Self-Consistency with sixteen samples, a test-time scaling baseline that, like LM-LGD, spends extra computation to improve a single model. Table D.10 reports the averages and the latency reduction of LM-LGD relative to that baseline. Because LM-LGD decodes the final answer once after a batched LM deliberation, whereas Self-Consistency runs sixteen full generations, LM-LGD is substantially faster on every benchmark while also being more accurate (Table 1). The reduction is largest on long-generation tasks such as MATH, where the cost of repeated decoding dominates.

Table D.10: Average per-example latency (seconds) on Qwen2.5-0.5B over ten random examples per benchmark. "Reduction" is the relative latency saving of LM-LGD over Self-Consistency (×16).

Benchmark	Self-Cons. (×16)	LM-LGD	Reduction
MMLU	4.2	1.6	61.9%
GSM8K	12.4	2.9	76.6%
HumanEval	15.1	3.1	79.5%
MATH	18.9	3.6	81.0%

The measured latencies match the analysis above. LM-LGD is more expensive than a single one-pass network, as Table 5 makes clear, but among methods that spend extra computation to raise accuracy it is comparatively efficient, because its cost grows with deliberation depth rather than with the number of generations or assistants. The reduction is largest on long-generation tasks such as MATH and HumanEval, where Self-Consistency pays for sixteen full decodes while LM-LGD decodes only once.

Appendix E. A detailed Walkthrough of the LM Deliberation Process

To complement the high-level description in Section 4.9, we trace a single forward pass of LM-LGD with its own symbols on a concrete question. We use an MMLU-style item that asks for the roots of a polynomial over a finite field:

> *Find all zeros in the indicated finite field of the given polynomial with coefficients in that field:* $x^3 + 2x + 2$ *in* $\mathbb{Z}_7$.

The correct answer is the set $\{2, 3\}$, since evaluating $p(x) = x^3 + 2x + 2 \bmod 7$ gives $p(2) = 14 \equiv 0$ and $p(3) = 35 \equiv 0$, while $p(x) \not\equiv 0$ for the remaining residues.

The box below traces the core operations of one forward pass on this question; the paragraphs that follow read the same steps in prose.

```
INPUT   : "zeros of p(x)=x^3+2x+2 over Z_7"     answer = {2,3}

[init]  H        = emb(x)                          # B x T x d
        h_i^0    = H + p_i      (p_i = 0 at init) # i = 1..N,  N = 6

[Phase 1: LM-level Deliberation]  for r = 0..R-1   (R = 3)
  for each receiver i, sender j != i:
    m_{j->i}^r = Edge_{j->i}(h_j^r + s_j + t_i)          # dense message
    a_{j->i}^r = Trust(pool(h_i^r), pool(m_{j->i}^r),
                       pool(H), p_i, p_j)                # scalar trust
    w_{j->i}^r = softmax_j(a_{j->i}^r)                   # convex weights
  c_i^r  = sum_j  w_{j->i}^r * m_{j->i}^r                # aggregated context
  g_i^r  = sigmoid(W_g [h_i^r ; c_i^r ; p_i])            # admit gate
  h_i^{r+1} = F(LayerNorm(h_i^r + g_i^r (*) c_i^r))      # shared backbone F
  # roles diverge: some track "mod 7", others evaluate p(x) at residues

[Phase 2: Reporting]
  rho_i = Router(pool(h_i^R), pool(H), p_i)              # score each assistant
```

```
  S     = top-k({rho_i}, k = 3)                          # delayed sparse select
  beta_i= softmax_{i in S}(rho_i)
  q     = sum_{i in S}  beta_i * W_rep h_i^R              # fused report
  g_c   = sigmoid(W_c [H ; q])
  H_core= LayerNorm(H + g_c (*) q)                       # inject into core node
  H_out = F(H_core)                                      # core reuses backbone F

OUTPUT  : decode(H_out W_lm^T)  ->  {2,3}
```

Input and initialization. The tokenized question x is embedded into $\mathbf{X} = \mathrm{Emb}(x) \in \mathbb{R}^{B\times T\times d}$. We instantiate N=6 assistants with $\mathbf{h}_i^0 = \mathbf{X} + \mathbf{p}_i$, where each $\mathbf{p}_i$ is zero-initialized, so at the first step every assistant holds exactly the pretrained representation of the question. Conceptually, the question carries several sub-tasks: identify the field $\mathbb{Z}_7$, parse the polynomial $p(x)$, and test candidate residues $x \in \{0, 1, \ldots, 6\}$.

Phase 1, round 1. Each ordered pair forms a message $\mathbf{m}_{j\to i}^0 = E_\phi^{j\to i}(\mathbf{h}_j^0 + \mathbf{s}_j + \mathbf{t}_i)$. A receiver i scores its incoming messages with $a_{j\to i}^0 = \mathrm{Trust}(\mathrm{pool}(\mathbf{h}_i^0), \mathrm{pool}(\mathbf{m}_{j\to i}^0), \mathrm{pool}(\mathbf{X}), \mathbf{p}_i, \mathbf{p}_j)$, normalizes them by softmax into $\alpha_{j\to i}^0$, and forms the context $\mathbf{c}_i^0 = \sum_{j\neq i} \alpha_{j\to i}^0 \mathbf{m}_{j\to i}^0$. It then updates $\mathbf{h}_i^1 = F_\theta(\mathrm{LayerNorm}(\mathbf{h}_i^0 + \mathbf{g}_i^0 \odot \mathbf{c}_i^0))$. In our reading of the trust and router statistics, the assistants begin to take on distinct sub-tasks at this point: some states emphasize the modulus and the reduction rule $a \bmod 7$, while others emphasize the arithmetic of evaluating $p(x)$ at particular residues.

Phase 1, rounds 2–3. The same update repeats for $r = 1, 2$. As messages circulate, an assistant that tentatively evaluated $p(2)$ can be corrected or confirmed by another that tracked the reduction modulo 7, and the trust weights sharpen so that each receiver leans on the few senders whose partial results it finds reliable. By the third round, the deliberated states $\{\mathbf{h}_i^3\}$ encode largely complementary pieces of the computation, for example the residues that satisfy $p(x) \equiv 0$ and those that do not, rather than six redundant copies of the same guess.

Phase 2, reporting and output. The router scores every assistant by $\rho_i = \mathrm{Router}(\mathrm{pool}(\mathbf{h}_i^3), \mathrm{pool}(\mathbf{X}), \mathbf{p}_i)$ and keeps the top k=3, fusing their projected reports as $\mathbf{q} = \sum_{i\in\mathcal{S}} \beta_i \mathbf{W}_{\mathrm{rep}} \mathbf{h}_i^3$ with β_i a softmax over the selected set. The fused report enters the core node through the gated residual $\mathbf{X}_{\mathrm{core}} = \mathrm{LayerNorm}(\mathbf{X} + \mathbf{g}_{\mathrm{core}} \odot \mathbf{q})$, and the core backbone produces $\mathbf{H}_{\mathrm{core}} = F_\theta(\mathbf{X}_{\mathrm{core}})$, from which the language-model head decodes the answer. For this question the decoded answer is the zero set $\{2, 3\}$. The walkthrough shows how the two phases divide the work: the LM deliberation phase lets several assistants explore the candidate residues and the modular reduction in parallel and correct one another, while the reporting phase commits only the most useful deliberated states to the core node, which produces the final result.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.

[2] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: International Conference on Learning Representations (ICLR), 2023.

[3] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: International Conference on Machine Learning (ICML), 2024.

[4] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, T. Hoefler, Graph of thoughts: Solving elaborate problems with large language models, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2024.

[6] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe, Let's verify step by step, in: International Conference on Learning Representations (ICLR), 2024.

[7] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, MetaGPT: Meta programming for a multi-agent collaborative framework, in: International Conference on Learning Representations (ICLR), 2024.

[10] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, J. Schmidhuber, GPTSwarm: Language agents as optimizable graphs, in: International Conference on Machine Learning (ICML), 2024.

[11] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, Y. Tian, Training large language models to reason in a continuous latent space, arXiv preprint arXiv:2412.06769 (2024).

[12] Z. Shen, H. Yan, L. Zhang, Z. Hu, Y. Du, Y. He, CODI: Compressing chain-of-thought into continuous space via self-distillation, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2025.

[13] S. Wu, Y. Wang, Q. Yao, Language model networks: Supervision-efficient learning through dense communication, arXiv preprint arXiv:2505.12741 (2025).

[14] Y. Xia, F. Tian, L. Wu, J. Lin, T. Qin, N. Yu, T.-Y. Liu, Deliberation networks: Sequence generation beyond one-pass decoding, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.

[15] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations (ICLR), 2018.

[16] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P. S. Yu, A comprehensive survey on graph neural networks, IEEE Transactions on Neural Networks and Learning Systems 32 (1) (2021) 4–24.

[17] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, in: International Conference on Learning Representations (ICLR), 2017.

[18] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, Journal of Machine Learning Research 23 (120) (2022) 1–39.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.

[20] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, et al., Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[21] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, et al., The llama 3 herd of models, arXiv preprint arXiv:2407.21783 (2024).

[22] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, Language models are unsupervised multitask learners, Tech. rep., OpenAI (2019).

[23] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, Journal of Machine Learning Research 21 (140) (2020) 1–67.

[24] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, T. B. Hashimoto, Stanford alpaca: An instruction-following LLaMA model, `https://github.com/tatsu-lab/stanford_alpaca` (2023).

[25] H. Kim, Y. Yu, L. Jiang, X. Lu, D. Khashabi, G. Kim, Y. Choi, M. Sap, ProsocialDialog: A prosocial backbone for conversational agents, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2022.

[26] M. Wu, A. Waheed, C. Zhang, M. Abdul-Mageed, A. F. Aji, LaMini-LM: A diverse herd of distilled models from large-scale instructions, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2024.

[27] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, J. Steinhardt, Measuring massive multitask language understanding, in: International Conference on Learning Representations (ICLR), 2021.

[28] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt, Measuring mathematical problem solving with the MATH dataset, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021.

[29] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman, Training verifiers to solve math word problems, arXiv preprint arXiv:2110.14168 (2021).

[30] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, arXiv preprint arXiv:2108.07732 (2021).

[31] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, W. Chen, MMLU-Pro: A more robust and challenging multi-task language understanding benchmark, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2024.

[32] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, J. Wei, Challenging BIG-Bench tasks and whether chain-of-thought can solve them, in: Findings of the Association for Computational Linguistics (ACL Findings), 2023.

[33] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, O. Tafjord, Think you have solved question answering? try ARC, the AI2 reasoning challenge, arXiv preprint arXiv:1803.05457 (2018).

[34] S. Lin, J. Hilton, O. Evans, TruthfulQA: Measuring how models mimic human falsehoods, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022.

[35] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, S. R. Bowman, GPQA: A graduate-level google-proof q&a benchmark, in: Conference on Language Modeling (COLM), 2024.

[36] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021).

[37] J. Novikova, O. Dušek, V. Rieser, The E2E dataset: New challenges for end-to-end generation, in: Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue, 2017.

[38] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[39] X. L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, in: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL), 2021.

[40] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, S. Gelly, Parameter-efficient transfer learning for NLP, in: International Conference on Machine Learning (ICML), 2019.